Integrated Automated Feeding and Watering System with Centralized Data Monitoring for Commercial Piggeries

An Undergraduate Thesis Proposal
Presented to the School of Engineering
University of San Carlos
Cebu City, Philippines

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Engineering

By

Alvin Joseph S. Macapagal

Jan Dave Q. Campañera

Vaun Michael C. Pace

Paul Andrew F. Parras

Lorenz Anthony L. Soriano

April 2026

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

ENDORSEMENT FOR PROPOSAL HEARING

Title of Thesis Proposal : **Integrated Automated Feeding and Watering System with Centralized Data Monitoring For Commercial Piggeries**

Name of Proponents : **Vaun Michael C. Pace**
Paul Andrew F. Parras
Lorenz Anthony L. Soriano

This is to certify that **we, the advisers, have reviewed and approved the project / research proposal** with the aforementioned title and proponents.

Therefore, we deemed this study **ready for proposal hearing** with the Evaluation Committee.

Hereby, we affix our signatures below as proof of endorsement.

EVALUATED BY:

ENGR. ALVIN JOSEPH S. MACAPAGAL
ADVISER

ENGR. JAN DAVE Q. CAMPAÑERA
ASSISTANT ADVISER

DATE EVALUATED: **28 APRIL 2026**

Integrated Automated Feeding and Watering System with Centralized Data Monitoring for Commercial Piggeries

Pace, Vaun Michael C.[1], Parras, Paul Andrew F.[1], Soriano, Lorenz Anthony L.[1],
Macapagal, Alvin Joseph S.[2], Campañera, Jan Dave Q.[2]
[1]*Undergraduate Student, Department of Computer Engineering, University of San Carlos, Talamban, Cebu City, Philippines*
[2]*Faculty Member, Department of Computer Engineering, University of San Carlos, Talamban, Cebu City, Philippines*
pacevaun@gmail.com, parraspaul13874@gmail.com, lorenzal.soriano@gmail.com,
ajsmacapagal@usc.edu.ph, jdcampanera@usc.edu.ph

***Abstract*—Manual feeding and inadequate health monitoring in commercial piggeries contribute to feed waste, undetected illness, and preventable livestock losses. This paper presents the design and development of an integrated, hardwired automated feeding and watering system with centralized data monitoring for commercial piggeries. The system employs a main control module connected to multiple feeder modules via a wired network, integrating weight sensors, water flow sensors, and air quality sensors to ensure precise feed rationing and continuous environmental monitoring. Collected data is processed and displayed through a local user interface, enabling caretakers to track consumption patterns and detect early signs of illness in real time. By eliminating reliance on manual observation and wireless communication, the system offers a reliable and scalable solution for improving biosecurity, animal health management, and operational efficiency in swine production.**

***Index Terms*—Livestock Automation, Precision Feeding, Biosecurity, Real-time Monitoring**

I. Statement of the Problem

Inconsistent feeding and drinking practices in pigs can lead to significant health risks and feed wastage [1]. Early detection of illness through physical or manual observation is often unreliable because subtle symptoms may go unnoticed [2]. Without a consistent feed and water tracking system, caretakers must rely on labor-intensive observation, which can lead to inaccurate records and delays in identifying symptoms or other issues. A pig's health status is closely linked to its feeding and drinking behavior, as measurable deviations in dietary intake are often among the first observable indicators of illness [3]. However, without a data-recording system, individual feeding patterns are not established or tracked over time, making it difficult to distinguish normal behavioral variation from significant decline. In addition, unmonitored air quality in piggery environments poses an unnecessary and preventable risk of common respiratory illnesses such as coughs and colds among pigs [4].

This study addresses these issues by proposing a system that establishes per-module consumption baselines using a rolling time window and flags deviations beyond a defined threshold as potential signs of abnormal behavior requiring immediate caretaker attention. Air-quality readings from integrated sensors are also compared against safe-range standards, with alerts generated when toxicity levels within the pig-pen environment exceed acceptable limits. This approach aims to reduce reliance on subjective visual observation and provide a more consistent and reliable basis for early health monitoring.

II. Statement of the Problem

Inconsistent feeding and drinking practices in pigs can lead to significant health risks and feed wastage [1]. Early detection of illness through physical or manual observation is often unreliable because subtle symptoms may go unnoticed [2]. Without a consistent feed and water tracking system, caretakers must rely on labor-intensive observation, which can lead to inaccurate records and delays in identifying symptoms or other issues. A pig's health status is closely linked to its feeding and drinking behavior, as measurable deviations in dietary intake are often among the first observable indicators of illness [3]. However, without a data-recording system, individual feeding patterns are not established or tracked over time, making it difficult to distinguish normal behavioral variation from significant decline. In addition, unmonitored air quality in piggery environments poses an unnecessary and preventable risk of common respiratory illnesses such as coughs and colds among pigs [4].

This study addresses these issues by proposing a system that establishes per-module consumption baselines using a rolling time window and flags deviations beyond a defined threshold as potential signs of abnormal behavior requiring immediate caretaker attention. Air-quality readings from integrated sensors are also compared against safe-range standards, with alerts generated when toxicity levels within the pig-pen environment exceed acceptable limits. This approach aims to reduce reliance on subjective visual observation and provide a more consistent and reliable basis for early health monitoring.

III. Goals and Objectives

The main goal of this project is to design and develop an integrated automated swine feeding and watering system with centralized data monitoring and control.

To achieve this goal, the system must fulfill the following specific objectives:

- Implement a programmable central control unit capable of real-time monitoring and control of feed storage, scheduled feed dispensing, water level monitoring, and automatic water dispensing when the level falls below the required threshold.
- Design individual feeder modules capable of monitoring feed consumption, water level, and ammonia levels, with defined validation baselines for feeding, water availability, and air-quality thresholds.
- Develop a centralized touchscreen user interface for system monitoring, data acquisition, historical logging, and graphical visualization of events relevant to final project checking.
- Integrate and validate a multi-channel alert system that notifies caretakers of consumption and environmental anomalies through the local dashboard and SMS notifications via a LoRa-GSM gateway, and evaluate its improvement over manual monitoring through controlled comparison, owner user-acceptance testing (UAT), Likert-scale confidence ratings, response-time measurements, and caretaker feedback.

IV. Scope and Limitations

This study focuses on the design and development of an automated feeding and watering system with data monitoring for commercial piggeries. The system is intended to support swine management by combining scheduled feed and water dispensing, sensor-based environmental monitoring, and digital record-keeping within a farrowing piggery prototype setup.

This study is limited to the following:

- The system does not perform direct veterinary diagnosis or laboratory disease confirmation.
- Implementation is limited to the system configuration used, with dimensions based on the farrowing pen setup; performance may vary across different farm layouts.
- System operation may be affected by interruptions in electrical power.
- Air quality monitoring is limited to sensors integrated within the feeder modules and covers only the immediate environment of each pen, not the entire piggery facility.
- Remote SMS alerts are limited by GSM signal availability, SIM card functionality, and mobile network service. During cellular service interruptions, alerts remain available through the local dashboard.
- The target thresholds for feeding, water level, and air quality will be established during calibration and baseline testing, and may require adjustment for different pig ages, pen sizes, feed types, trough geometry, ventilation conditions, and farm-management practices.
- Improvement over manual labor will be evaluated using controlled comparison with manual monitoring, time-and-motion logs, caretaker response-time measurements, owner UAT, and Likert-scale feedback. These measures evaluate operational support and user confidence, not veterinary diagnosis.
- Final deployment validation depends on access to a local partner piggery or farm owner who permits installation, observation, and UAT. If full deployment is not allowed, the study will document the limitation and conduct partner-observed bench or pilot testing using the same event-recording protocol.

V. Significance of the Study

This study holds significant value for multiple sectors, ranging from small-scale backyard farmers to major commercial piggery operators, as well as broader agricultural and technological stakeholders.

Swine Farmers and Caretakers

By replacing manual feeding with a sensor-based dispensing and monitoring system, the proposed solution directly addresses feeding problems such as inconsistent rationing, excess feed distribution, feed wastage, and difficulty tracking actual feed intake per pen. The system measures the amount of feed dispensed and consumed through the load cell, allowing caretakers to compare current consumption with previous feeding records. When feed intake drops below the established baseline, the system flags the change as a feeding irregularity that requires inspection. In this way, the system helps improve feeding consistency, reduce feed waste, and provide caretakers with a clearer basis for identifying pens that need attention.

Animal Health and Biosecurity

The proposed system provides caretakers with continuous records of feed consumption, water level, and air-quality conditions in each monitored pen. These records allow caretakers to compare current pen conditions with previous readings instead of relying only on manual observation. When the system detects abnormal feed consumption, low water level, or air-quality readings beyond the set threshold, it generates an alert for caretaker inspection.

Through automated alerts and structured data records, caretakers can respond more promptly to feeding irregularities, water supply issues, or unsafe air-quality conditions. These alerts provide caretakers with timestamped information on the type and location of the detected abnormal condition, allowing them to inspect the affected pen based on recorded sensor readings rather than manual observation alone. In this way, the system supports organized record-keeping and consistent monitoring of feeding, watering, and environmental conditions.

The Philippine Swine Industry

With approximately 71.1% of the Philippine hog inventory managed by smallholder backyard farmers, the local swine sector remains highly vulnerable to disruptions caused by disease and inefficient farm management. A reliable, centralized automated system could help stabilize domestic pork production, reduce dependence on imported meat, which reached a record 1.64 million metric tons in 2025, and support the national goal of rebuilding the country's swine population to pre-ASF levels.

Field of Agricultural Automation and Embedded Systems

This study contributes to the growing body of research on IoT and precision agriculture applied to livestock management. By demonstrating a fully hardwired, sensor-integrated feeding and monitoring system designed specifically for the Philippine piggery context, the project provides a practical reference for future engineering work in agricultural automation, particularly in environments where wireless reliability may be a concern.

Future Researchers

The system's data logging and analytics capabilities generate structured, timestamped records of feed consumption, water intake, and environmental conditions. This dataset can serve as a foundation for future studies on swine behavioral patterns, machine learning-based health prediction models, and the broader integration of precision livestock farming technologies in developing agricultural economies.

VI. Review of Related Literature

A. Automated Feeding and Watering Systems

Automated feeding and watering systems in commercial piggeries are designed to deliver controlled and consistent amounts of feed and water using sensor-based monitoring and programmable control logic. These systems aim to reduce feed wastage and improve efficiency by replacing manual, labor-dependent practices with precise and repeatable operations [5]–[8].

Literature emphasizes that inconsistencies in feeding and drinking behavior can lead to health deterioration and inefficient resource utilization [1], [4], [9]. As a result, automated systems are designed to standardize feeding schedules and quantities while ensuring that consumption is monitored continuously. A key consideration in system design is maintaining consistency across multiple feeding points, which requires coordinated control and reliable data exchange within the system [7], [8].

Gravimetric dispensing—wherein load cells measure the actual mass delivered per feeding event—has been identified in the literature as a practical approach to achieving high accuracy in automated feed delivery [8], [10]. By measuring dispensed mass directly rather than relying on volumetric or timed dispensing, systems can compensate for variations in feed bulk density and auger flow characteristics. The use of auger screw conveyors introduces inherent variability due to feed moisture content, particle size, and mechanical wear; for this reason, a tolerance of $\pm 5\%$ (corresponding to a dispensing accuracy of $\geq 95\%$) is commonly applied as an operational benchmark in precision feeding systems [5], [10]. Water availability management is equally emphasized in the literature: research indicates that changes in water intake are among the earliest observable indicators of health problems in swine, making automated water level monitoring and refill control an integral component of modern feeding systems [9], [11], [12].

B. Health Monitoring Through Behavioral and Consumption Data

Behavioral monitoring through feeding and drinking patterns is widely recognized as a non-invasive method for early disease detection in swine. Changes in consumption behavior often occur before visible clinical symptoms, making them valuable indicators of potential health issues [3], [13].

Studies such as Dingcong et al. demonstrate that tracking feeder and drinker interactions can provide insight into animal health and behavior [3]. More broadly, research highlights the importance of establishing baseline consumption patterns and identifying deviations over time [13], [14]. These deviations can be analyzed using time-series approaches, where historical data is used to define normal behavior and detect anomalies.

A rolling-window approach to anomaly detection—wherein the system computes a moving mean and standard deviation over a defined number of recent sessions and flags current consumption that falls outside a statistical band—has been reported as effective for detecting health-related intake changes in swine [13], [14]. This method accommodates natural day-to-day variation while remaining sensitive to behaviorally significant declines, reducing false-positive alert rates compared to fixed-threshold approaches [13].

Without structured data collection and analysis, caretakers rely on subjective observation, which may result in delayed detection of illness. The literature supports the use of continuous monitoring systems to provide objective and data-driven assessment of animal health conditions [3], [14].

C. Air Quality Sensing in Swine Environments

Environmental conditions within piggery enclosures play a significant role in animal health and productivity. Ammonia (NH_3), produced primarily from the microbial decomposition of urine and feces, is the dominant respiratory hazard in enclosed swine facilities [15], [16]. The Occupational Safety and Health Administration (OSHA) has established 25 ppm as the permissible exposure limit (PEL) for ammonia in occupational settings, a threshold that is widely adopted as a practical alarm level in swine confinement monitoring systems [15], [16]. Research indicates that ammonia concentrations exceeding 20–25 ppm can impair the mucociliary defenses of the respiratory tract in pigs, increasing susceptibility to secondary bacterial infections and reducing overall productivity [13], [15].

Poor air quality has been associated with respiratory illnesses and reduced overall welfare in swine populations [4], [17]. Water availability is also a critical management variable

because changes in water intake can signal performance and health problems in swine herds [11], [12].

Research indicates that monitoring environmental parameters enables early identification of unsafe conditions and supports timely intervention. Air quality data also complements behavioral monitoring, as environmental stressors can influence feeding and drinking patterns [13], [17]. This relationship highlights the importance of integrating environmental and behavioral data to achieve a more comprehensive understanding of animal health.

Existing studies also explore advanced approaches such as sound-based detection of respiratory symptoms, further reinforcing the role of non-invasive monitoring techniques in livestock management [17].

D. Wired Communication for Embedded Livestock Systems

The selection of a reliable wired communication protocol is a critical design decision for multi-node embedded monitoring systems deployed in agricultural environments. RS-485 with the MODBUS RTU protocol is widely established as a suitable backbone for such systems due to its differential signaling, high noise immunity in electrically harsh environments, and support for bus topologies spanning up to 1,200 meters [5], [7]. These properties are particularly relevant in metal-framed piggery structures where electromagnetic interference from motorized equipment can degrade wireless signals.

In precision livestock farming, wired communication provides a more deterministic and fault-tolerant data path than wireless alternatives, which are susceptible to multipath fading and signal blockage in enclosed agricultural structures [5], [18]. A MODBUS RTU master-slave architecture, wherein a central controller sequentially polls each sensor node at defined intervals, enables predictable latency and straightforward fault isolation—both of which are important for maintaining the data integrity required for anomaly detection systems [7], [18].

E. Centralized Data Monitoring and User Interface Design

Centralized monitoring systems are widely used in agricultural automation to aggregate data from multiple sensing points and provide a unified view of system status. These systems enable continuous data logging, alert generation, and historical trend visualization, supporting informed decision-making by farm operators [6], [14], [19], [20].

The literature emphasizes the importance of accessible and intuitive user interfaces that present complex data in a simplified manner. Features such as dashboards with near-real-time updates, historical trend visualization, and threshold-based alerts are commonly identified as essential components of effective monitoring systems [13], [19], [20]. Near-real-time data display—typically characterized by end-to-end latency on the order of seconds—is considered sufficient for farm health monitoring applications, where the primary goal is to detect trends rather than respond to instantaneous events [6], [18].

For embedded livestock monitoring platforms, continuous availability is a key performance requirement: systems that experience significant downtime lose monitoring coverage during which health or environmental anomalies may go undetected. Research on IoT-based precision livestock farming systems identifies $\geq 95\%$ operational availability as a baseline benchmark for farm deployment [5], [18]. In addition, centralized systems reduce reliance on manual observation and improve the accuracy and consistency of recorded data. Continuous monitoring technologies have been shown to enhance productivity and reduce labor demands in modern swine farming operations [4], [21].

F. Disease Outbreaks and Farm-Level Monitoring in Swine Production

Disease outbreaks such as African Swine Fever have emphasized the importance of biosecurity and consistent farm-level monitoring in swine production. However, automated monitoring systems do not replace veterinary diagnosis or laboratory confirmation. Their role is to support daily farm management by continuously recording feeding behavior, water availability, and environmental conditions that may indicate abnormal pen conditions requiring caretaker inspection. In this context, the proposed system contributes to biosecurity-oriented management by improving the consistency of monitoring and record-keeping, rather than by directly detecting or preventing ASF [22].

G. Scalability and Deployment Considerations in Philippine Piggeries

The structure of the Philippine swine industry, characterized by a combination of commercial farms and smallholder operations, presents unique challenges for technology adoption. With a large proportion of hog production managed by backyard farmers, systems must be adaptable, scalable, and accessible [21].

Literature highlights the need for flexible system designs that can accommodate varying farm sizes and operational conditions. Continuous monitoring technologies are increasingly being adopted in modern farming environments, demonstrating their potential to improve productivity and animal welfare [4], [21].

Additionally, non-invasive monitoring approaches, including sound-based and image-based techniques, provide scalable methods for observing animal health across large populations [3], [23]. These approaches expand the range of available tools for precision livestock farming and support the gradual modernization of the sector.

VII. Methodology

This chapter presents the conceptual framework underpinning the system's design philosophy, followed by a detailed account of the system architecture, hardware construction, feed hopper and trough volume calculations, feed and water dispensing mechanism, environmental and consumption sensing strategy, wired communication protocol, and system performance evaluation metrics.

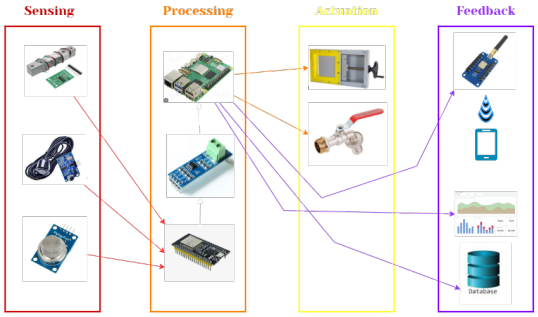


Fig. 1. Conceptual Framework

A. Conceptual Framework

As illustrated in Figure 1, the system is conceptualized as a closed-loop precision livestock management platform grounded in two theoretical underpinnings: Precision Agriculture Theory—which frames sensor-driven, individualized resource delivery as the basis for efficient swine production—and Feedback Control Systems Theory, which models each feeder module as a node whose output (feed and water dispensed) is continuously logged and compared against defined consumption baselines to flag deviations indicative of illness [6], [13].

The system's operational logic spans four functional layers:

- **Layer 1 – Sensing (Input):** Each feeder module integrates a load cell with an HX711 24-bit ADC for gravimetric feed measurement, a waterproof ultrasonic sensor (e.g., JSN-SR04T) for water-level monitoring, and an MQ-135 electrochemical sensor for ammonia air-quality monitoring [11]–[13].
- **Layer 2 – Processing and Decision (Control):** Each feeder module's ESP32 microcontroller evaluates sensor readings against pre-established rolling-window consumption baselines. Deviations beyond a defined threshold are flagged as health alerts and relayed to the main control module via the RS-485 wired bus. The Raspberry Pi central controller processes incoming data, executes alert logic, and manages the touchscreen dashboard.
- **Layer 3 – Actuation (Output):** Upon a scheduled or manually triggered feeding event, the main control module runs the auger motor and opens the solenoid cutoff valve located at the target feeder module's trough inlet for a calibrated duration, dispensing a gravimetrically verified quantity of feed into the trough. Positioning the cutoff valve at the feeder module rather than at the hopper ensures that any feed already in the conveyor pipe is fully delivered before the valve closes, preventing waste and reducing residual blockage in the pipe. Water is dispensed through a solenoid valve when the sensor detects the water level has dropped below a configurable refill threshold. This threshold is set by the caretaker or farm owner through the touchscreen dashboard and is stored in the SQLite database; if no custom value has been configured, the system defaults to 40% of the calibrated trough depth. An active-high push button on pin RA0 provides a hardware-level manual override that is always accessible to the caretaker regardless of the pen's current alert status. Pressing RA0 navigates the dashboard to the Manual Override screen, where the caretaker can choose to manually dispense a specified quantity of feed (in kilograms) or water (in liters) without waiting for a scheduled cycle.
- **Layer 4 – Feedback, Logging, and Remote Notification:** All sensor readings, dispensing events, and alerts are timestamped and stored in a local SQLite database on the Raspberry Pi. In addition to displaying alerts on the local touchscreen dashboard, the system utilizes a LoRa (Long Range) transceiver to transmit critical health and environmental alerts to a remote LoRa-GSM gateway positioned outside the piggery. Upon receiving the LoRa payload, the gateway forwards the alert as an SMS message to the caretaker's mobile phone, enabling remote notification even when the caretaker is away from the main control module [6], [23].

B. System Architecture: Distributed Software Workflow

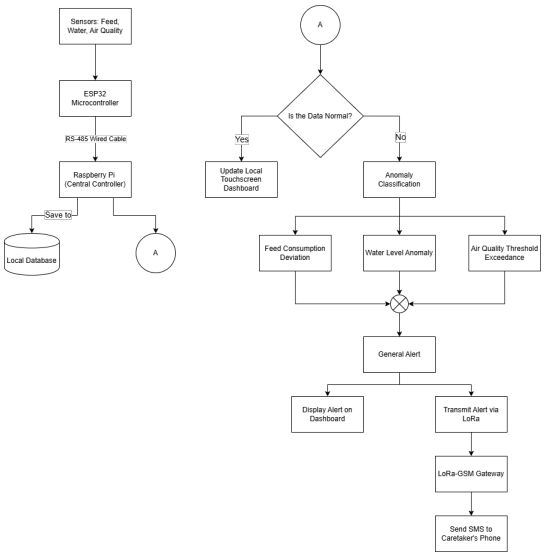


Fig. 2. System Flowchart

The system employs a distributed, hierarchical software architecture to decouple real-time hardware interfacing from high-level data orchestration [5], [6]. As illustrated in Figure 2, the software workflow operates in a continuous loop. Data acquisition begins at the edge nodes where the ESP32 samples the sensors via hardware interrupts. The Raspberry Pi systematically polls these nodes over the RS-485 bus. Upon receiving the telemetry, the Python backend writes the raw data to the SQLite database and passes it to the Analytical Engine. If the data falls within normal baseline parameters, it is pushed to the Flask frontend for live dashboard visualization. If an anomaly is detected, the system simultaneously triggers a visual alert on the local touchscreen dashboard and transmits

an alert payload via LoRa to the remote LoRa-GSM gateway, which forwards it as an SMS to the caretaker's mobile phone.

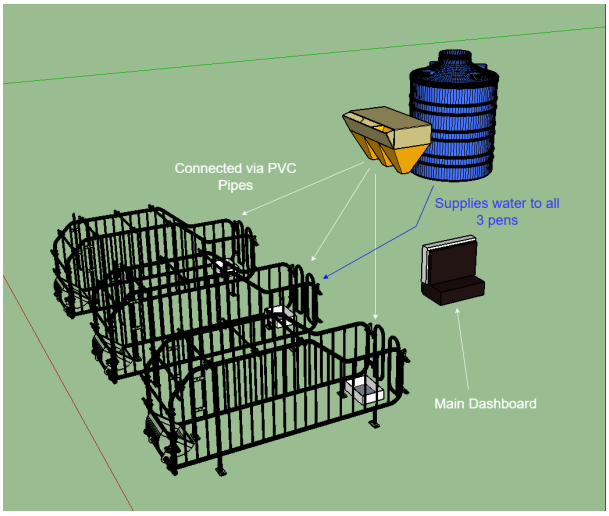


Fig. 3. 3D Visualization of the System Deployment in a Piggery Environment

Figure 3 illustrates the intended physical deployment of the system within a commercial piggery environment. Three individual pen enclosures, each equipped with a feeder module containing the load cell, ultrasonic water sensor, and MQ-135 air quality sensor, are connected to a centralized water supply tank via PVC piping. The central feed hopper and main control module (Raspberry Pi with the touchscreen dashboard) are positioned adjacent to the pen structures, providing the caretaker with direct physical access to the monitoring interface while maintaining proximity to the RS-485 wired bus connecting all feeder nodes.

- **Edge Intelligence (Feeder Nodes):** Each node is governed by an ESP32 dual-core microcontroller. One core is dedicated to continuous sensor polling (load cell) using hardware interrupts and the HX711 24-bit ADC, while the second core manages a MODBUS RTU slave stack. Telemetry is stored in a local register map, allowing the master to retrieve data asynchronously without blocking sensor acquisition.
- **Central Orchestration (Main Module):** A Raspberry Pi serves as the MODBUS RTU master, implementing a sequential polling loop via the RS-485 bus. The main module is co-located with the central hopper—a 0.18 m^3-capacity truncated-pyramid storage unit—from which feed is gravity-fed through an auger conveyor to individual feeder nodes. The master logic is divided into four primary services:
  - **Poller Service:** Executes MODBUS read commands at 1-second intervals to aggregate per-pen telemetry.
  - **Analytical Engine:** Implements the rolling-window anomaly detection logic, comparing incoming data against the SQLite-persisted baseline.
  - **Command Service:** Dispatches metered dispensing instructions based on the pre-configured feeding schedule or manual overrides.
  - **LoRa Alert Service:** Interfaces with a connected LoRa transceiver module via SPI or UART. When the Analytical Engine flags an anomaly based on the rolling-window baseline, this service encodes the alert data into a lightweight payload and transmits it over the LoRa link to the remote LoRa-GSM gateway for SMS forwarding to the caretaker's mobile phone.
- **Remote LoRa-GSM Gateway:** A secondary edge device positioned outside the piggery in a location with adequate cellular coverage. It receives incoming LoRa alert payloads and forwards them as SMS messages to the caretaker's registered mobile phone number via an onboard GSM module, enabling immediate caretaker notification without requiring physical presence at the main control module.

C. Software Architecture and Technology Stack

The system's software architecture is distributed across the edge feeder nodes and the central control module. The technology stack was selected to prioritize fault tolerance, low-latency execution, and complete offline capability, ensuring that the system remains fully operational without external internet dependencies. The end-to-end data flow pipeline, from sensor acquisition to dashboard visualization, is depicted in Figure 4.

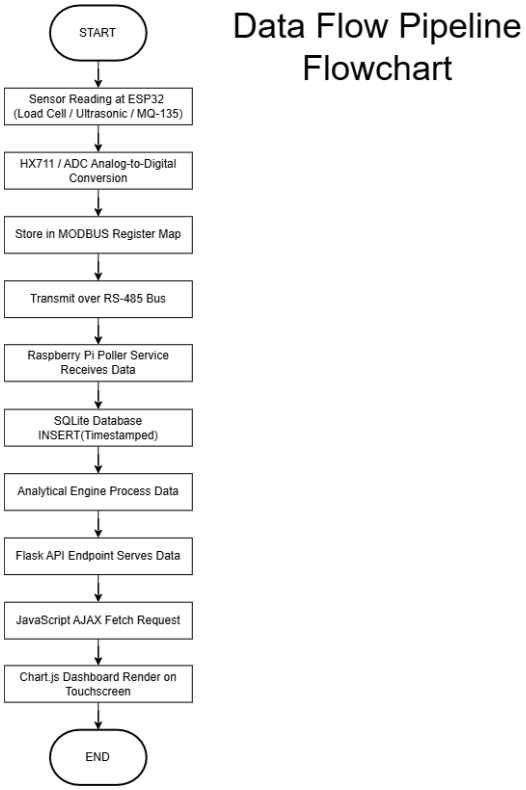


Fig. 4. Data Flow Pipeline Flowchart

The end-to-end data flow pipeline is illustrated in Figure 4, tracing the path of a single sensor reading from acquisition at the ESP32 edge node through ADC conversion, RS-485 transmission, SQLite persistence, analytical processing, and final visualization on the touchscreen dashboard [19], [20].

1. Edge Node Firmware (ESP32)

- **Software/Language:** C/C++ utilizing the FreeRTOS (Real-Time Operating System) environment.
- **Why it is used:** C/C++ provides the low-level memory management and fast execution speeds required for high-frequency sensor sampling. FreeRTOS allows the ESP32 to effectively utilize its dual-core processor by dividing tasks into concurrent threads.
- **How it is implemented:** As shown in Figure 5, Core 0 is dedicated strictly to hardware interrupts—specifically reading the HX711 ADC—minimizing the risk of missed sensor readings during execution delays. Core 1 manages the MODBUS RTU slave stack, listening for polling requests from the Raspberry Pi and transmitting the localized sensor data stored in its memory registers.

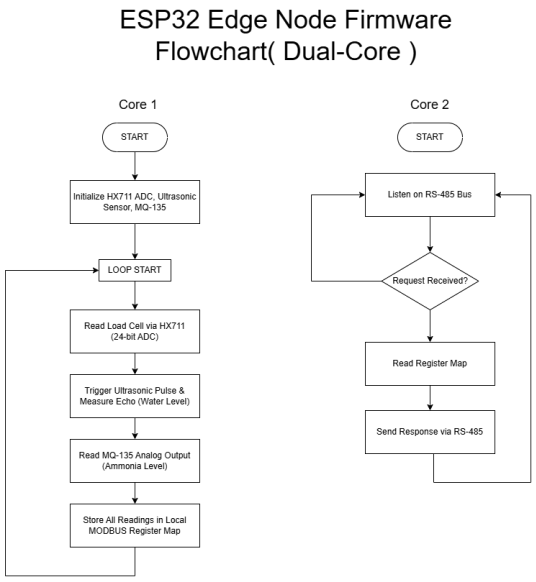


Fig. 5. ESP32 Edge Node Firmware Flowchart (Dual-Core)

The dual-core firmware architecture is depicted in Figure 5. Core 0 continuously cycles through sensor acquisition (load cell, ultrasonic, and MQ-135 readings), while Core 1 concurrently handles MODBUS RTU communication on the RS-485 bus.

2. Central Controller Backend (Raspberry Pi)

- **Software/Language:** Python 3, Flask Web Framework, and PyModbus.
- **Why it is used:** Python offers extensive library support for serial communication and data analysis. Flask is a lightweight WSGI web application framework that does not require heavy dependencies, making it ideal for an embedded Linux environment. PyModbus provides a highly stable, synchronous protocol for RS-485 communication.
- **How it is implemented:** Python acts as the central orchestrator. A background script runs continuous PyModbus loops to query each ESP32 node. The retrieved data is passed through the Python-based Analytical Engine to calculate rolling averages and detect anomalies. Simultaneously, Flask serves the backend API, exposing this data to the local network so the frontend can retrieve it dynamically without requiring a page refresh.

3. Data Persistence (Database)

- **Software:** SQLite.
- **Why it is used:** Unlike heavy relational databases (e.g., MySQL or PostgreSQL) that require dedicated server processes, SQLite is a C-language library that implements a small, fast, self-contained, high-reliability SQL database engine. It writes directly to ordinary disk files, minimizing CPU and RAM overhead on the Raspberry Pi.
- **How it is implemented:** The Python backend executes standard SQL commands to log timestamped telemetry data, system alerts, and feeding schedules. To prevent SD card degradation from excessive write cycles, database commits are batched and executed at defined intervals rather than instantaneously per sensor read.

4. User Interface and Visualization (Frontend)

- **Software/Language:** HTML5, CSS3, vanilla JavaScript, and Chart.js.
- **Why it is used:** This combination supports a responsive dashboard suited to routine farm monitoring. Chart.js is specifically utilized because it renders lightweight, HTML5 canvas-based time-series graphs that do not lag on embedded processors [19], [20].
- **How it is implemented:** The Raspberry Pi boots directly into a Chromium browser in ”Kiosk Mode” (fullscreen, no navigation bars), pointing to the local Flask server address. JavaScript asynchronous requests (AJAX) continuously pull the latest SQLite data via Flask API endpoints, dynamically updating the Chart.js line graphs and numeric alert gauges so caretakers see real-time shifts in consumption and air quality.

D. Volume and Structural Calculations

The physical dimensions of the central hopper and feeder trough were derived from pig-feed bulk density and a total required storage capacity of 90 kg, using a bulk density of 550 kg/m^3. The minimum required storage volume is:

$$V = \frac{m}{\rho} = \frac{90 \text{ kg}}{550 \text{ kg/m}^3} \approx 0.1636 \text{ m}^3 \tag{1}$$

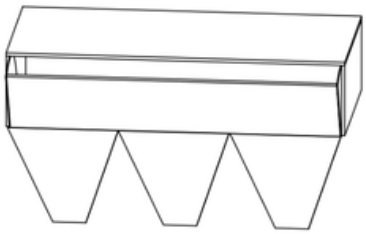


Fig. 6. Main Control Module

An allowance of $0.180\ \text{m}^3$ was adopted as the design target. The overall assembly of the main control module is shown in Figure 6. The central storage hopper is designed as a truncated rectangular pyramid (frustum) with a top face of $L = 1.0\ \text{m} \times W = 0.5\ \text{m}$ and a bottom opening of $a = b = 0.1\ \text{m}$, which connects directly to the auger inlet. The slant height of the frustum walls is calculated from:

$$h = \frac{(0.5\ \text{m} - 0.1\ \text{m})}{2} \times \tan(60^\circ) \approx 0.35\ \text{m} \tag{2}$$

The volume of each conical frustum section is given by the prismatoid formula, with corresponding dimensions illustrated in Figure 7:

$$V_{cone} = \frac{h}{3}(A_B + a_b + \sqrt{A_B \cdot a_b}) \tag{3}$$

where $A_B = 0.5\ \text{m} \times 0.333\ \text{m} = 0.165\ \text{m}^2$ is the top-face area per section, and $a_b = 0.1\ \text{m} \times 0.1\ \text{m} = 0.01\ \text{m}^2$ is the bottom-aperture area. Substituting into (3):

$$V_{cone} \approx 0.02516\ \text{m}^3 \times 3 \approx 0.075\ \text{m}^3 \tag{4}$$

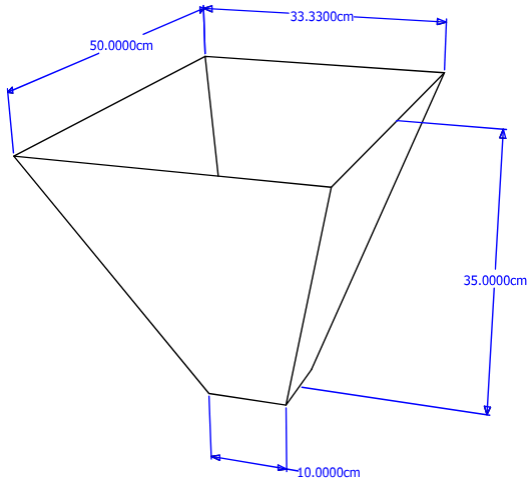


Fig. 7. Cone Dimensions

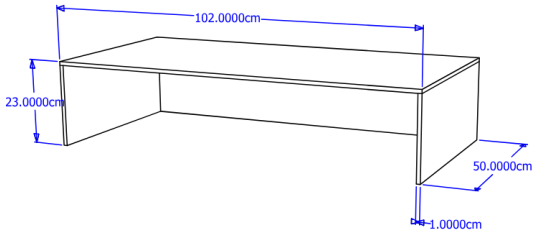


Fig. 8. Box Dimensions

The remaining volume is provided by a rectangular upper reservoir section of height h_{rec}, with the corresponding box dimensions shown in Figure 8. This is computed from:

$$V_{rec} = V_{total} - V_{cones} = 0.180 - 0.075 = 0.105\ \text{m}^3 \tag{5}$$

Solving for the rectangular section height with $L = 1.0\ \text{m}$ and $W = 0.5\ \text{m}$:

$$h_{rec} = \frac{0.105\ \text{m}^3}{1.0\ \text{m} \times 0.5\ \text{m}} = 0.21\ \text{m} \tag{6}$$

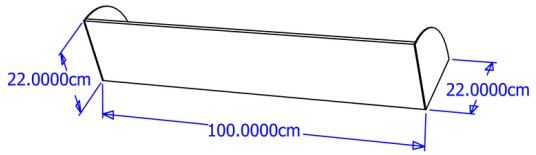


Fig. 9. Hopper Lid Dimensions

The total hopper height is therefore $h_{cone} + h_{rec} = 0.35 + 0.21 = 0.56$ m, housed within an overall footprint of 102 cm $\times$ 50 cm at the top rim, as shown in Figure 9. The hopper lid is manually opened by the caretaker for feed replenishment. A separate funnel outlet is provided for each feeder node, directing feed into the auger conveyor.

E. Dispensing Mechanism

Feed flows by gravity from the central hopper through the $0.1\ \text{m} \times 0.1\ \text{m}$ bottom aperture directly into the auger screw conveyor, which meters delivery through the feed pipe to each feeder module. At each feeder module's trough inlet, a 12 V DC solenoid cutoff valve controls the final release of feed into the trough, as illustrated in Figure 11. Positioning the cutoff valve at the feeder module rather than at the hopper ensures that any feed already in the conveyor pipe is fully delivered before the valve closes, preventing waste and reducing residual blockage in the pipe. This valve type was selected for its simplicity, low power consumption, and compatibility with granular feed materials.

An auger screw conveyor driven by a NEMA 17 stepper motor (shown in Figure 10) serves as the primary metering mechanism. By counting motor steps, the ESP32 can dispense feed in calibrated increments independently of gravity head pressure, improving dose accuracy across varying hopper fill levels.

Fig. 10. NEMA 17 Stepper Motor

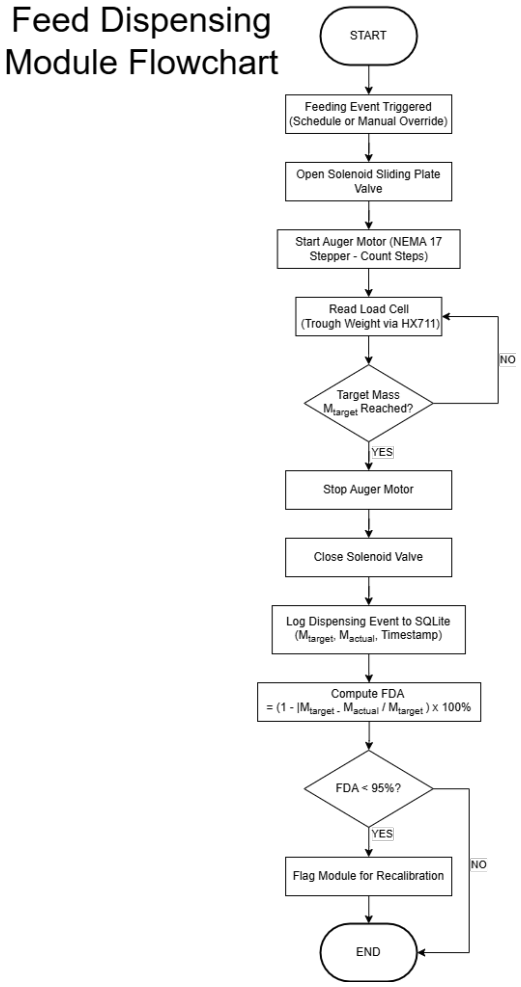


Fig. 11. Feed Dispensing Module Flowchart

The feed dispensing process is summarized in Figure 11. When a feeding event is triggered, the system runs the auger motor and opens the solenoid cutoff valve at the target feeder module. The load cell continuously reads the trough weight until the target mass is reached, at which point the auger motor is stopped, the cutoff valve is closed, and the event is logged.

F. Sensing Strategy and Component Selection

Sensor selection was guided by the operating environment of a commercial piggery: high ambient humidity, a corrosive ammonia atmosphere, mechanical vibration from pig activity, and the need for low-cost scalability across multiple feeder nodes.

- **Feed Weight (Load Cell + HX711):** As shown in Figure 12, a strain-gauge load cell rated for the expected trough load is interfaced to an HX711 24-bit sigma-delta ADC. The HX711 communicates with the ESP32 over a dedicated two-wire serial interface, providing 10 or 80 samples per second. The trough weight is logged before and after each feeding event; the delta constitutes the dispensed mass. Consumption is derived by tracking weight loss between feeding cycles.

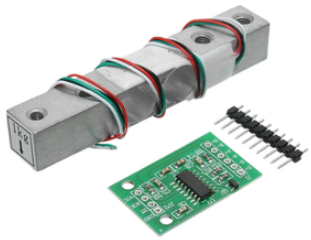


Fig. 12. HX711 & Straight Bar Load Cell

Fig. 13. JSN-SR04T Waterproof Ultrasonic Sensor

- **Water Availability (Level Sensor):** As depicted in Figure 13, a waterproof ultrasonic sensor is mounted above each pen's water trough. Each per-pen water trough has a maximum holding capacity of 5 liters. The ESP32 triggers an ultrasonic pulse and measures the echo time of flight to calculate the distance to the water surface. This determines the current water level as a percentage of the trough's 5-liter capacity and triggers the solenoid valve to refill the trough when the level falls below a configurable refill threshold. This threshold is not hardcoded; it is set by the caretaker or farm owner through the dashboard's Water Refill Threshold settings screen and persisted in the SQLite database. The system defaults to 40% of the

5-liter capacity (equivalent to 2.0 L) if no value has been explicitly configured. This design allows the threshold to be adjusted to suit different pig categories, trough geometries, and farm-specific operational requirements without modifying the firmware. The complete water-refill control loop is illustrated in Figure 14 [7], [11], [12].

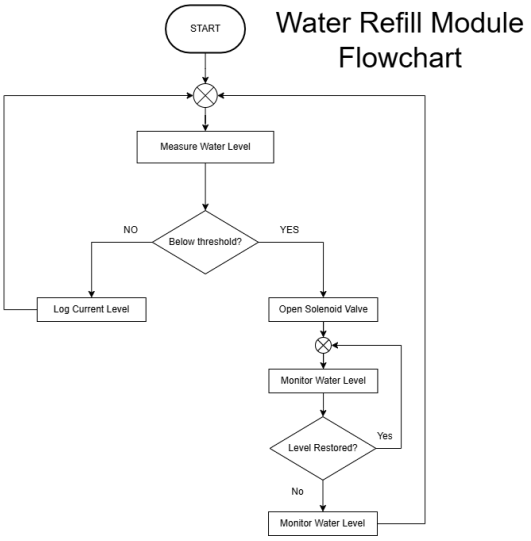


Fig. 14. Water Refill Module Flowchart

The water-refill process is illustrated in Figure 14. The system continuously measures the water level; when it drops below the caretaker-configured threshold, the solenoid valve opens and the level is monitored until it is restored, at which point the valve closes and the event is logged [11], [12].

Fig. 15. MQ-135 Air Quality Sensor

- **Air Quality (MQ-135):** The MQ-135 electrochemical sensor, shown in Figure 15, detects ammonia (NH_3), CO_2, and benzene vapors—the primary respiratory hazards in piggery environments arising from urine decomposition. The sensor's analog output is read by the ESP32's onboard ADC and compared against safe-range thresholds. A default alert threshold of 25 ppm is adopted, consistent with the OSHA permissible exposure limit and the practical alarm level recommended for swine confinement facilities in the literature [15], [16]; exceedances trigger caretaker alerts on the dashboard.

G. Wired Communication: RS-485 / MODBUS RTU

The system specifies a hardwired network topology to reduce dependence on wireless signal integrity in metal-framed piggery structures. RS-485 with the MODBUS RTU protocol was selected as the communication backbone for the following reasons [5], [6]:

- Differential signaling on a twisted-pair bus provides high noise immunity in electrically harsh farm environments.
- The standard supports up to 32 nodes on a single bus segment at distances of up to 1,200 m, enabling future expansion of feeder modules without architectural redesign.
- MODBUS RTU is an industry-standard protocol with extensive ESP32 and Raspberry Pi library support, reducing firmware development overhead.

Each feeder-node ESP32 is interfaced to the bus via a MAX485 half-duplex transceiver IC. The Raspberry Pi master polls each slave node at configurable intervals, retrieves sensor data packets, and writes dispensing commands, as illustrated in Figure 16. A unique MODBUS device address is assigned to each feeder module during commissioning.

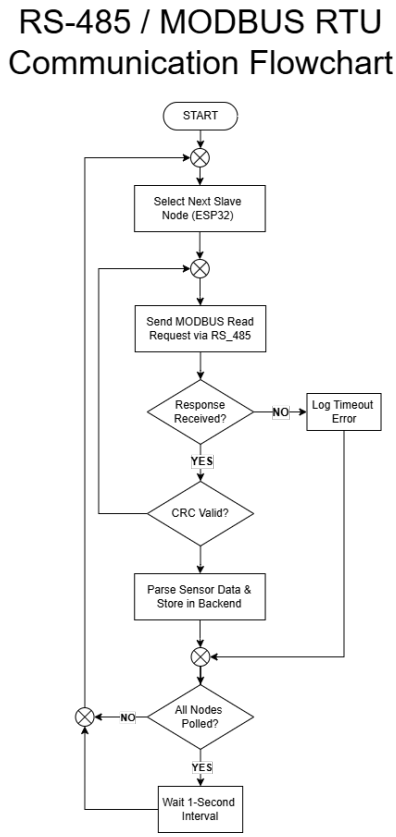


Fig. 16. RS-485 / MODBUS RTU Communication Flowchart

The polling protocol is depicted in Figure 16. For each cycle, the master selects the next slave node, transmits a MODBUS read request, validates the response CRC, and parses

the sensor data before moving to the next node. Timeout and retry logic ensures robustness against transient communication failures.

H. Application Layer and Data Persistence

The centralized user interface (UI) is hosted on the Raspberry Pi and rendered through a 7-inch DSI touchscreen mounted on the main control module enclosure. The software stack utilizes a lightweight web-based architecture to ensure low-latency updates:

- **Backend Stack:** A Flask-based web server manages the API endpoints for the dashboard. It interfaces with an SQLite database for ACID-compliant persistence of timestamped consumption logs, feeding schedules, and air-quality telemetry.
- **Frontend Stack:** A Chromium kiosk browser displays a dashboard built with responsive numeric gauges and time-series charts for real-time monitoring and historical trend visualization. It includes configuration panels for feeding schedules and alert logs for threshold exceedances.
- **Fault Tolerance:** All data is stored locally, ensuring continuous operation during internet outages, which is a primary concern identified in the study's scope and limitations. To address potential grid instability, a UPS module with 18650 lithium cells provides bridge power, ensuring that the SQLite database maintains integrity during short-duration power interruptions.
- **LoRa-SMS Alert Gateway:** A background Python thread runs alongside the Flask backend, with the end-to-end alert pipeline depicted in Figure 17. It continuously monitors the SQLite database for newly generated threshold exceedances. To optimize radio duty cycles and prevent packet collisions, the gateway implements a message-queuing protocol, bundling non-critical alerts and transmitting them via the LoRa link to the remote LoRa-GSM gateway at configured intervals or triggering immediate LoRa transmission for critical air-quality hazards. The remote gateway then forwards each alert as an SMS to the caretaker's mobile phone.

The end-to-end alert notification pipeline is illustrated in Figure 17. When the Analytical Engine flags an anomaly, the alert payload is encoded and transmitted via LoRa to the remote gateway, which decodes it and forwards it as an SMS to the caretaker's mobile phone.

I. Dashboard User Interface Design

The touchscreen dashboard is designed as a multi-screen web application rendered in Chromium kiosk mode on the Raspberry Pi's 7-inch DSI display. The interface follows a hierarchical navigation pattern that allows caretakers to progress from a system-wide overview to per-pen diagnostics and configuration with minimal interaction. The five primary screens are described below.

1. Main Dashboard (System Overview)

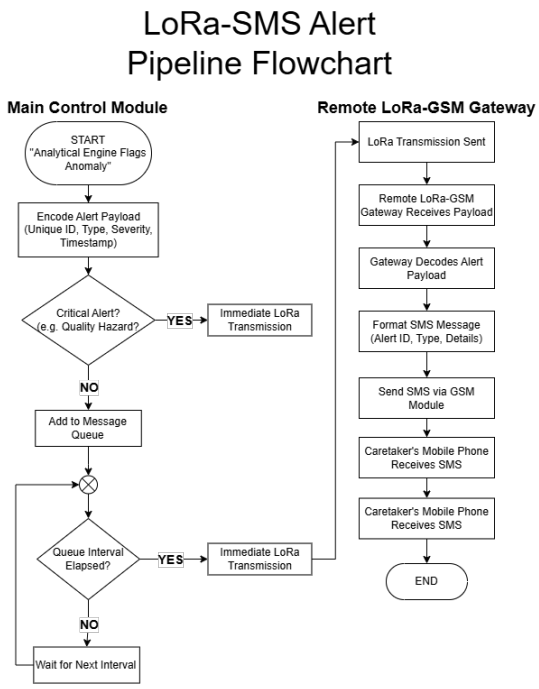


Fig. 17. LoRa-SMS Alert Pipeline Flowchart

Fig. 18. Main Dashboard Screen

The main dashboard, shown in Figure 18, serves as the system's landing screen and provides a consolidated, at-a-glance overview of all connected feeder modules. Each pen is represented as an individual card displaying four key real-time parameters: water level (as a percentage bar), feed remaining (in kilograms), ammonia (NH_3) concentration (in ppm), and a color-coded status indicator. The status indicator uses a three-tier alert scheme:

- **Green (No Active Alerts):** All sensor readings are within normal baselines.
- **Yellow (Warning):** A non-critical threshold has been exceeded (e.g., rising ammonia levels approaching the 25 ppm limit).
- **Red (Critical):** A critical threshold has been breached

(e.g., water level below the configured refill threshold), requiring immediate caretaker attention.

The caretaker can tap on any pen card to navigate to the detailed Pen Control screen for that specific module.

2. Pen Control (Per-Pen Detailed View)

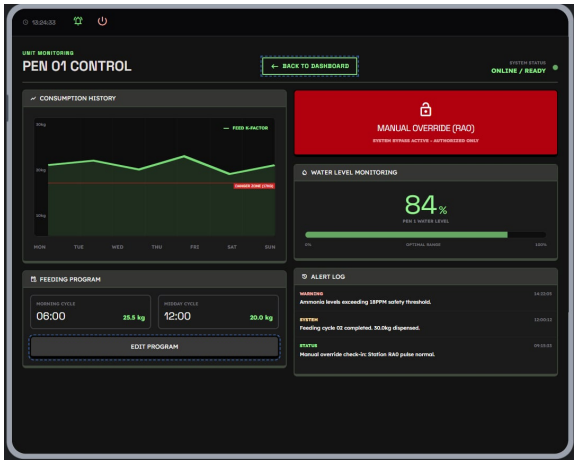


Fig. 19. Pen Control Screen

Upon selecting a pen card from the main dashboard, the system navigates to the Pen Control screen, shown in Figure 19. This screen presents a comprehensive data breakdown for the selected feeder module, organized into the following functional panels:

- **Consumption History:** A time-series line chart (rendered via Chart.js) displaying the pen's daily feed consumption over the past week. A horizontal "Danger Zone" threshold line marks the lower bound of normal consumption, enabling the caretaker to visually identify declining intake trends that may indicate illness.
- **Water Level Monitoring:** A large numeric gauge displaying the current water trough level as a percentage, along with the currently configured auto-refill threshold. An "Edit Refill Threshold" button provides direct access to the Water Refill Threshold settings screen.
- **Feeding Program:** A summary panel showing the currently configured feeding schedule, including the start time and target dispensing mass (in kilograms) for each feeding cycle (e.g., Morning Cycle at 06:00, Midday Cycle at 12:00). An "Edit Program" button provides direct access to the Feed Settings screen for schedule modification.
- **Alert Log:** A chronological list of recent system events and alerts for the selected pen, categorized by severity (Warning, Critical, Status) and timestamped for traceability.
- **Manual Override (RA0) Button:** A prominently displayed button that is always accessible from the Pen Control screen, regardless of the pen's current alert status. Tapping this button navigates the caretaker to the Manual Override screen, where an on-demand dispensing command can be issued. This button reflects the hardware-level RA0 override push button and provides its software equivalent on the dashboard.

A "Back" button in the navigation bar returns the user to the main overview screen.

3. Feed Settings (Schedule Configuration)

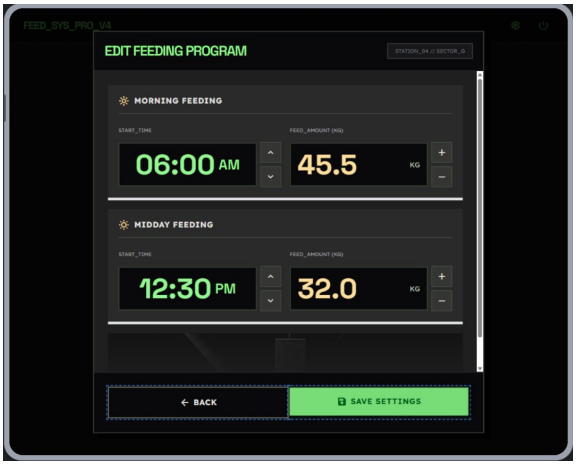


Fig. 20. Feed Settings Screen

The Feed Settings screen, shown in Figure 20, is accessed by tapping the "Edit Program" button on the Pen Control screen. This screen allows the caretaker to configure the automated feeding schedule for the selected pen. Each feeding cycle (e.g., Morning Feeding, Midday Feeding) is presented as an independent configuration block containing two adjustable parameters:

- **Start Time:** The scheduled dispensing time, adjustable via up/down arrow controls in hours and minutes (AM/PM format).
- **Feed Amount (kg):** The target mass to be dispensed during the cycle, adjustable via increment/decrement buttons.

Once the desired values are set, the caretaker taps "Save Settings" to persist the updated schedule to the SQLite database, which the Command Service references for subsequent automated dispensing events. A "Back" button returns the user to the Pen Control screen without saving changes, preventing accidental schedule modifications.

4. Water Refill Threshold (Water Level Configuration)

The Water Refill Threshold screen is accessed by tapping the "Edit Refill Threshold" button on the Pen Control screen. This screen allows the caretaker or farm owner to set the water level percentage at which the system will automatically open the solenoid valve to refill the trough. The configurable threshold is stored per pen in the SQLite database and is referenced by the water-refill control loop during normal automated operation. The current threshold is displayed on the Pen Control screen for immediate reference. If no value has been explicitly set by the caretaker, the system uses a default threshold of 40% of the calibrated trough depth. The screen also displays a note recommending that the threshold be validated against manual depth measurements before deployment and adjusted to match the pig category, trough geometry, and

observed daily water consumption. A "Save Threshold" button persists the updated value, and a "Back" button returns to the Pen Control screen without saving.

5. Manual Override (RA0) Dispensing Screen

The Manual Override screen is accessible via the "Manual Override (RA0)" button on the Pen Control screen. Unlike the status-indicator behavior described in earlier prototype iterations, this button is always present and active regardless of the pen's alert state, reflecting the fact that the RA0 push button is a hardware-level control mechanism available to the caretaker at any time. Upon navigating to this screen, the caretaker is presented with two independent dispensing options:

- **Dispense Feed:** The caretaker sets the desired feed quantity in kilograms using increment and decrement controls, then taps "Dispense Feed Now." The Command Service issues an immediate dispensing instruction to the auger motor, which runs until the load cell confirms the target mass has been delivered to the trough.
- **Dispense Water:** The caretaker sets the desired target fill level as a percentage using increment and decrement controls, then taps "Dispense Water Now." The Command Service opens the solenoid valve and monitors the ultrasonic level sensor until the trough water level reaches the specified target, at which point the valve closes automatically. No scheduled time is required for either operation; the dispensing command executes immediately upon confirmation.

Both override events are timestamped and logged in the SQLite database as manual override entries, ensuring complete traceability in the event log. A "Back" button returns the caretaker to the Pen Control screen.

J. Consumption Baseline and Anomaly Detection

A pig's health status is closely linked to its feeding and drinking behavior; measurable deviations in dietary intake are often among the first observable indicators of illness [3], [13]. The system establishes per-module baselines using a rolling time-window approach consistent with data-driven livestock monitoring methods reported in the literature [13], [14]. Let C_i denote the feed consumed in session i. To avoid using the current observation as part of its own baseline, the rolling mean for the current session t is computed from the previous N completed sessions:

$$\mu_{t-1,N} = \frac{1}{N} \sum_{i=t-N}^{t-1} C_i \tag{7}$$

A deviation alert is generated when the current session's consumption C_t falls outside the band $[\mu_{t-1,N} - k\sigma_{t-1,N}, \mu_{t-1,N} + k\sigma_{t-1,N}]$, where $\sigma_{t-1,N}$ is the rolling standard deviation of the same previous N sessions and k is a configurable sensitivity coefficient (default $k = 2$). This statistical approach reduces false positives arising from natural day-to-day variation while reliably flagging behaviorally significant declines [13], [14].

In practice, this algorithm is executed by the Analytical Engine running on the Raspberry Pi, as illustrated in Figure 21. After each feeding session, the Python backend queries the SQLite database to retrieve the most recent previous $N = 14$ consumption records for the corresponding feeder module. The rolling mean $\mu_{t-1,N}$ and standard deviation $\sigma_{t-1,N}$ are then computed in memory. If the current session's consumption C_t falls below $\mu_{t-1,N} - k\sigma_{t-1,N}$, the system classifies the event as a *feed consumption deviation*; if C_t exceeds $\mu_{t-1,N} + k\sigma_{t-1,N}$, it is flagged as an *abnormal overconsumption event*. In parallel, water-level readings are compared against calibrated trough-level thresholds, while MQ-135 air-quality readings are converted to estimated ammonia concentration and compared against baseline and warning thresholds. When any anomaly is detected, the Analytical Engine simultaneously pushes a visual alert to the Flask-served touchscreen dashboard and dispatches a LoRa payload to the remote LoRa-GSM gateway for SMS delivery to the caretaker. All anomaly events, including their type, severity, timestamp, threshold values, baseline values, and acknowledgement status, are persisted in the SQLite database for historical trend analysis and final project checking.

Target Threshold Baselines for Validation. During commissioning, each feeder module will undergo a silent baseline period before health alerts are activated. The following initial validation thresholds will be used as starting targets and will be adjusted with the farm owner or caretaker according to the pig category, trough geometry, ventilation condition, and normal feeding schedule:

- **Feeding baseline:** The normal feed-consumption baseline for each pen is the previous $N = 14$ completed feeding sessions. A warning is generated when $C_t < \mu_{t-1,N} - 2\sigma_{t-1,N}$ or when consumption falls by at least 20% below the rolling mean for two consecutive scheduled feedings. A critical alert is generated when consumption falls by at least 30% below the rolling mean or when no measurable feed reduction is detected after a scheduled feeding interval.
- **Water-level baseline:** Each water trough has a maximum holding capacity of 5 liters, which corresponds to 100% on the dashboard's water level gauge. The ultrasonic sensor is calibrated by measuring the empty (0 L), minimum acceptable, and full (5 L) trough depths with a manual ruler or dipstick. For validation, the normal operating band is set at 50–100% of the calibrated trough depth (2.5–5.0 L). The auto-refill threshold is configurable by the caretaker or farm owner through the dashboard's Water Refill Threshold settings screen; if no custom value has been set, the system defaults to 40% (equivalent to 2.0 L). A critical alert is generated when the level remains below 30% (1.5 L) for more than 5 minutes after the refill command is issued, or when the sensor reading deviates by more than ± 1 cm from the reference measurement during calibration. Because this parameter is not fixed in firmware, the caretaker should validate the

configured threshold with manual depth measurements before full deployment and adjust it according to the specific trough geometry, pig category, and daily water consumption patterns observed at the farm.

- **Air-quality baseline:** Before deployment, the MQ-135 module is burned in, calibrated in clean air to establish R_0, and logged for at least 24 hours in the target pen area to determine the normal background ammonia estimate. The initial interpretation bands are: normal at < 10 ppm, caution at 10–19 ppm, warning at 20–24 ppm, and critical at ≥ 25 ppm. The ≥ 25 ppm level is treated as an indication of potentially unsafe or unsanitary pen conditions requiring cleaning or ventilation inspection, not as a veterinary diagnosis. The threshold will be validated by comparing sensor readings before cleaning, after cleaning, and after ventilation changes with caretaker observations of manure accumulation, wet bedding or flooring, and odor intensity.

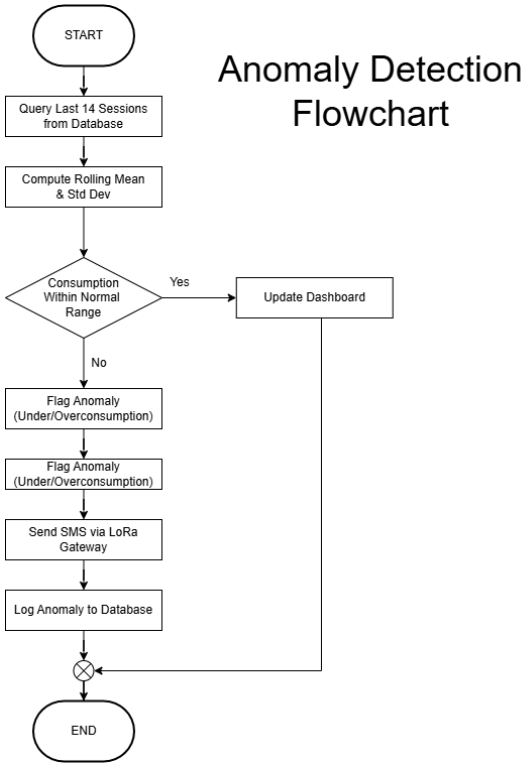


Fig. 21. Anomaly Detection Flowchart

The anomaly detection logic is summarized in Figure 21. After each feeding session, the system queries the previous 14 completed sessions, computes the rolling mean and standard deviation, and determines whether the current consumption falls within the normal range. If an anomaly is flagged, alerts are simultaneously pushed to the dashboard and transmitted via the LoRa-SMS pipeline.

K. Evaluation Metrics and Verification Protocol

System performance will be assessed against quantitative metrics aligned with the project's specific objectives. These metrics are intended to reflect the reliability of dispensing, sensing, communication, and health-monitoring functions commonly emphasized in precision livestock and embedded monitoring studies [5], [6], [13]. To ensure system reliability and accuracy prior to full-scale deployment, a rigorous testing protocol will be executed to systematically validate these metrics.

Quantitative Metrics:

- **Feed Dispensing Accuracy (FDA):** Percentage agreement between the target dispensed mass and the gravimetrically measured actual dispensed mass across a calibration trial of 30 dispensing events per module. A minimum passing threshold of $\geq 95\%$ is adopted, consistent with accuracy benchmarks reported in comparable automated feeding and precision livestock systems [3], [5], [24]. This threshold was selected because a $\pm 5\%$ tolerance accounts for the inherent variability of granular feed flow through auger mechanisms while remaining within nutritionally acceptable dispensing limits for swine rations [10], [25]:

$$FDA = \left(1 - \frac{|M_{target} - M_{actual}|}{M_{target}}\right) \times 100\% \quad (8)$$

  In the system, M_{target} is the pre-configured feed mass (in grams) specified in the feeding schedule stored in the SQLite database, while M_{actual} is the gravimetrically measured mass recorded by the load cell after each dispensing event. For each of the 30 calibration trials, the Python backend logs both values and computes the per-event FDA. The overall FDA for a module is reported as the mean across all trials. If the mean FDA falls below 95%, the system flags the module for recalibration of the auger stepper motor step-to-mass conversion factor.
- **Alert Response Accuracy (ARA):** Proportion of simulated consumption anomaly events correctly detected and flagged by the baseline algorithm within one polling cycle. A minimum passing threshold of $\geq 90\%$ is required to validate the reliability of the anomaly detection logic [13], [14]:

$$ARA = \frac{\text{True Positive Alerts}}{\text{Total Induced Events}} \times 100\% \quad (9)$$

  During validation, the tester deliberately induces consumption anomalies by withholding or reducing the feed placed in the trough, simulating a sick pig's reduced intake. Each induced event is logged manually alongside the system's automated response. A *True Positive* is counted when the Analytical Engine correctly flags the induced deviation within one polling cycle (≤ 1 second). Any induced event that does not trigger an alert is classified as a *False Negative*. The ARA is then computed as the ratio of correctly detected events to total induced events across all test runs.
- **System Uptime (U_{sys}):** Measured over a 72-hour continuous operation trial, consistent with endurance validation practices for embedded monitoring systems [5],

[6]. A minimum threshold of $\geq 95\%$ is adopted, in line with availability benchmarks commonly applied to IoT-embedded agricultural monitoring systems, where continuous sensor coverage is required but brief, recoverable interruptions (e.g., software restarts) are acknowledged as acceptable [6], [18], [26]. At 95% over a 72-hour trial, this permits a maximum cumulative downtime of approximately 3.6 hours—a ceiling beyond which monitoring gaps become operationally significant in a farm setting:

$$U_{sys} = \frac{T_{total} - T_{downtime}}{T_{total}} \times 100\% \tag{10}$$

where T_{total} is the total trial duration in hours and $T_{downtime}$ is the cumulative time during which sensor polling or the dashboard was unavailable. In the system, a Python watchdog thread periodically writes a heartbeat timestamp to the SQLite database. $T_{downtime}$ is computed post-trial by identifying gaps in the heartbeat log that exceed a predefined timeout interval (e.g., > 10 seconds), and summing their durations.

- **Data Display Latency:** Time elapsed between a sensor reading being acquired at the feeder node and its appearance on the dashboard, measured across 50 consecutive poll cycles. Target threshold: ≤ 2 seconds, consistent with near-real-time data display benchmarks reported in IoT-based swine monitoring systems [6], [27].
- **SMS Delivery Rate (SDR):** The end-to-end percentage of alert messages successfully delivered as SMS to the caretaker's mobile phone via the LoRa-GSM gateway, compared to the total number of alerts generated by the central unit. A minimum passing threshold of $\geq 95\%$ is required to confirm reliable remote notification delivery [6], [23]:

$$SDR = \frac{\text{Total SMS Delivered}}{\text{Total Alerts Generated}} \times 100\% \tag{11}$$

During testing, the system logs each alert event with a unique identifier in the SQLite database at the moment of LoRa transmission. The caretaker's mobile phone is used to count the total number of SMS messages received, each of which contains the corresponding alert identifier. The SDR is then computed by dividing the number of confirmed SMS receipts by the total number of LoRa-transmitted alerts logged in the database.

- **End-to-End Alert Latency:** The time elapsed between an alert being generated by the Analytical Engine and the corresponding SMS being successfully delivered to the caretaker's mobile phone, encompassing LoRa transmission, gateway processing, and GSM delivery. Target threshold: ≤ 30 seconds per alert message [6], [23].
- **Manual Labor Reduction (MLR):** Percentage reduction in caretaker monitoring time when using the system compared with manual observation. The baseline manual-monitoring time T_{manual} will be measured by observing the caretaker's normal process for checking feed level, water availability, air condition, and record entry. The assisted monitoring time T_{system} will be measured while the caretaker uses the dashboard and alert log for the same tasks:

$$MLR = \frac{T_{manual} - T_{system}}{T_{manual}} \times 100\% \tag{12}$$

A target reduction of at least 25% will be used as the initial validation threshold, subject to confirmation with the farm owner during UAT.

- **Owner/Caretaker User Acceptance Score (UAS):** Mean Likert-scale rating from owner and caretaker UAT forms covering UI/UX clarity, confidence in sensor readings, alert usefulness, ease of changing feeding schedules, ease of acknowledging alerts, and perceived labor reduction. Each item will be scored from 1 (strongly disagree) to 5 (strongly agree):

$$UAS = \frac{\sum_{j=1}^{Q} S_j}{Q} \tag{13}$$

The system passes UAT when the mean score is ≥ 4.0 and no critical usability item scores below 3.

- **Event-Record Completeness (ERC):** Percentage of required project-checking events with complete timestamped records, including scheduled feeding, manual override, refill command, low-water alert, feed-consumption deviation, ammonia warning or critical alert, SMS transmission, SMS receipt, caretaker acknowledgement, and corrective action:

$$ERC = \frac{\text{Complete Event Records}}{\text{Required Event Records}} \times 100\% \tag{14}$$

The target threshold is $ERC \geq 99\%$ during the final checking period.

L. Testing and Checking

The testing protocol is designed to systematically verify that each of the project's four specific objectives has been accomplished. Each test is explicitly mapped to its corresponding objective, and the pass/fail criteria are defined such that successful completion of all tests constitutes a complete demonstration of the system's intended functionality. This structured approach ensures that panelists and evaluators can trace every test result directly to the objective it validates [3], [5], [6].

Controlled Testing Environment. All tests described in this section are designed to be conducted entirely in a controlled laboratory or bench-testing environment, without requiring deployment to an actual piggery. This allows evaluators and panelists to assess the full system performance during a demonstration session. The real-world conditions of a piggery are simulated as follows:

- **Feed consumption simulation:** Before testing, the feeder module will be calibrated using a reference weighing scale to verify the load cell readings. A known amount of feed will then be placed in the trough and recorded as the initial feed weight. During normal consumption simulation, predetermined quantities of feed will be removed to represent expected intake based on the selected

pig category or test specimen. To simulate abnormal consumption, a lower amount of feed will be removed, or no feed will be removed, to represent reduced intake. The system reading before and after each removal will be compared with the reference scale reading to determine if the module accurately records feed consumption and flags abnormal intake.

- **Water level simulation:** Each water trough has a maximum holding capacity of 5 liters, corresponding to 100% on the dashboard gauge. The trough will be filled to its full 5-liter capacity and recorded as the baseline reading. Water will then be drained in measured amounts (e.g., 0.5 L increments) to simulate drinking behavior. For abnormal water-level simulation, the water level will be reduced below the configured refill threshold faster than normal to verify the automatic solenoid valve trigger. The system reading before and after each draining step will be recorded and compared with the expected level to verify whether the automatic refill response and alert trigger operate correctly.
- **Air quality simulation:** The MQ-135 sensor will be placed inside a controlled test enclosure or near a contained ammonia vapor source at a fixed distance. Baseline air-quality readings will first be recorded under normal air conditions. A diluted ammonia solution or controlled ammonia vapor source will then be introduced gradually to raise the sensor reading above the configured threshold bands: normal (< 10 ppm), caution (10–19 ppm), warning (20–24 ppm), and critical (≥ 25 ppm). The system will record the before-and-after readings and verify whether the dashboard and alert system correctly identify the air-quality anomaly. The source will then be removed to confirm whether the reading returns toward normal levels.
- **Baseline and Reference Values:** The baseline for feed and water consumption will be established using recorded normal readings during the initial testing period. For validation, each test will include before-and-after measurements. Feed readings from the load cell will be compared with a calibrated weighing scale, while water-level readings will be compared with the known initial and drained water levels. Ammonia readings will be compared against the configured threshold used by the system. These reference values will determine whether the system reading is accurate and whether the alert was correctly triggered.
- **LoRa-SMS notification simulation:** The LoRa-GSM gateway is positioned at a measured distance from the main control module within the test venue. Anomalies are induced as described above, and the evaluator verifies SMS receipt on the caretaker's registered mobile phone in real time.
- **Continuous operation simulation:** For the 72-hour uptime and data persistence tests, the system is run on a benchtop with periodic automated or manual sensor interactions to generate representative data volumes, verifying that the database and dashboard remain operational over extended periods.
- **Control-group comparison:** A manual-monitoring group will perform the same checking tasks using the current caretaker process: visual feed inspection, water-level inspection, air-condition observation or odor check, handwritten log entry, and manual notification. A system-assisted group will perform the same tasks using the touchscreen dashboard, event log, and SMS alerts. Completion time, missed anomalies, record completeness, and response time will be compared between the two groups.
- **Owner UAT and Likert-scale confidence survey:** The farm owner or designated caretaker will perform guided UAT tasks: log in to the dashboard, inspect each pen card, interpret a warning and a critical alert, modify one feeding schedule, acknowledge an alert, perform a manual override, and review historical logs. After the task set, the owner/caretaker will answer a 5-point Likert form covering UI/UX clarity, confidence in feeding readings, confidence in water-level readings, confidence in air-quality warnings, usefulness of SMS alerts, ease of schedule editing, and perceived labor reduction.
- **Final-checking event record:** Every final-checking run will use an event checklist that records the event identifier, date and time, feeder module, induced or observed condition, expected system response, actual dashboard response, SMS status, caretaker action, evaluator signature, and owner/caretaker remarks. This checklist provides the traceable evidence that the project objectives were tested.

This controlled approach ensures that every aspect of the system—dispensing accuracy, sensor reliability, anomaly detection, dashboard functionality, labor-reduction potential, owner acceptance, and remote SMS notification—can be validated by evaluators. If a local partner piggery permits pilot installation, the same protocol will be repeated in the deployment area; otherwise, the controlled demonstration will be documented as a bench validation with partner-observed UAT.

Objective 1: Implement a programmable central unit to manage main storage and automate the dispensing process.

This objective requires demonstrating that the central unit (Raspberry Pi and hopper assembly) can store feed, execute a programmable feeding schedule, and dispense accurate quantities of feed to each feeder module without manual intervention.

- **Test 1.1 – Feed Dispensing Accuracy (FDA):** The auger-based dispensing mechanism will be tested across 30 dispensing events per feeder module. For each event, the target mass M_{target} is set via the dashboard's feeding schedule, and the actual dispensed mass M_{actual} is measured by the load cell. The FDA for each event is computed as:

$$FDA = \left(1 - \frac{|M_{target} - M_{actual}|}{M_{target}}\right) \times 100\% \quad (15)$$

Pass Criterion: Mean FDA across all 30 trials $\geq 95\%$. This threshold is adopted from accuracy benchmarks reported in comparable automated feeding and precision livestock systems, including auger-based pig feeding systems that have demonstrated dispensing errors as low as 0.25% average and 1.5% maximum [3], [5], [24]. A $\pm 5\%$ tolerance was selected because it accounts for the inherent variability of granular feed flow through auger mechanisms—which can exhibit discharge variability of approximately 5% due to hopper–conveyor coupling parameters—while remaining within nutritionally acceptable dispensing limits for swine rations [10], [25].
How this demonstrates the objective: If the system consistently dispenses feed within $\pm 5\%$ of the programmed target across repeated trials, it confirms that the central unit can reliably automate the dispensing process with the accuracy required for precision feeding. The use of 30 trials ensures statistical robustness and accounts for natural variability in granular feed flow [3], [5].

- **Test 1.2 – Schedule Execution Verification:** A 24-hour automated feeding schedule consisting of three feeding events at pre-configured times will be programmed into the dashboard. The system must autonomously initiate each dispensing event within ± 30 seconds of the scheduled time without manual intervention.
  Pass Criterion: All three scheduled feeding events are executed within the time tolerance window, and the corresponding dispensed masses are logged in the SQLite database with correct timestamps.
  How this demonstrates the objective: Successful execution of all scheduled events proves that the central unit is fully programmable and can manage the dispensing process autonomously over a sustained period, fulfilling the core requirement of automated feed management.

Objective 2: Design individual feeder modules capable of monitoring feed consumption, water availability, and ammonia levels.

This objective requires demonstrating that each feeder module's sensors produce accurate, reliable readings for feed weight, water level, and air quality under realistic conditions.

- **Test 2.1 – Load Cell Calibration and Accuracy:** Each load cell and HX711 assembly will be calibrated using standard reference weights across a range of 0 to 5 kg. A minimum of 10 calibration trials per load cell will be conducted at five weight increments (1 kg, 2 kg, 3 kg, 4 kg, 5 kg). Linearity (R^2) and hysteresis will be evaluated.
  Pass Criterion: Linear regression $R^2 \geq 0.99$ across the tested range, and all individual readings within $\pm 2\%$ of the reference weight.
  How this demonstrates the objective: Accurate load cell readings are the foundation for feed consumption monitoring. A high R^2 confirms that the sensor produces linear, repeatable measurements, ensuring that the feeder module can reliably track how much feed a pig consumes per session [3], [5].
- **Test 2.2 – Water Level Sensor Validation:** The ultrasonic level sensors will be validated against manual depth measurements using a calibrated dipstick across a range of operational trough depths (5 cm to 25 cm). A minimum of 10 trials per sensor will be conducted. The auto-refill threshold used during this test will be the caretaker-configured value stored in the SQLite database, or the system default of 40% if no custom value has been set.
  Pass Criterion: All sensor readings within ± 1 cm of the manual dipstick measurement, and the solenoid valve correctly triggers when the water level falls below the configured or default refill threshold.
  How this demonstrates the objective: Accurate water level readings confirm that the feeder module can reliably monitor water availability. Correct solenoid triggering at the caretaker-defined threshold proves the module can maintain adequate water supply under variable operational parameters without manual intervention [5], [6].
- **Test 2.3 – Air Quality Sensor Calibration (MQ-135):** The MQ-135 sensors will undergo a minimum 48-hour burn-in period followed by calibration in a controlled, clean-air environment to establish a baseline resistance (R_0). Sensors will then be tested against known ammonia concentrations in the range of 10 to 300 ppm to verify threshold response accuracy.
  Pass Criterion: The sensor correctly triggers an alert when ammonia concentration exceeds the configured threshold (default: 25 ppm) in at least 9 out of 10 controlled exposure trials.
  How this demonstrates the objective: Reliable threshold detection confirms that the feeder module can monitor ammonia levels and identify hazardous air quality conditions, fulfilling the environmental monitoring requirement of this objective [5], [13], [17].
- **Test 2.4 – Threshold Baseline Validation:** The initial feeding, water-level, and air-quality thresholds will be validated before final testing. For feeding, at least 14 normal feeding sessions per module will be logged to establish $\mu_{t-1,N}$ and $\sigma_{t-1,N}$. For water, the empty, minimum acceptable, and full trough depths will be measured manually and mapped to percentage readings. For air quality, ammonia readings will be logged before cleaning, during intentionally dirty or high-odor pen conditions when available, after cleaning, and after ventilation changes.
  Pass Criterion: Feeding thresholds correctly classify at least 90% of induced normal/reduced-consumption cases; water readings remain within ± 1 cm of manual depth measurements; and air-quality alerts follow the established bands of normal (< 10 ppm), caution (10–19 ppm), warning (20–24 ppm), and critical (≥ 25 ppm). Owner or caretaker observations of dirty, wet, or poorly ventilated pen conditions must be recorded beside each air-quality reading.
  How this demonstrates the objective: This test provides

explicit baseline thresholds for validation and links ammonia toxicity readings to practical sanitation indicators, such as manure accumulation, wet flooring, odor, and ventilation status.

Objective 3: Develop a user interface on the central unit for system monitoring, data acquisition, and historical logging.

This objective requires demonstrating that the touchscreen dashboard displays real-time sensor data, allows configuration of feeding schedules, and maintains a persistent historical log accessible to the caretaker.

- **Test 3.1 – Data Display Latency:** The time elapsed between a sensor reading being acquired at the feeder node and its appearance on the touchscreen dashboard will be measured across 50 consecutive poll cycles.
  Mean latency ≤ 2 seconds, with no individual reading exceeding 3 seconds.
  How this demonstrates the objective: A mean latency of ≤ 2 seconds, with no individual reading exceeding 3 seconds, confirms that the dashboard provides near-real-time monitoring with sufficiently low delay for practical farm supervision, ensuring the caretaker sees current data rather than stale readings. IoT-based swine monitoring systems have demonstrated dashboard response times of under 1 second in production environments [27], validating that this threshold is both achievable and appropriate for livestock health monitoring. This validates the system monitoring component of the objective [5], [6].
- **Test 3.2 – Data Persistence and Historical Logging:** The system will be operated continuously for 72 hours. After the trial, the SQLite database will be queried to verify that all sensor readings (feed weight, water level, ammonia), dispensing events, and anomaly alerts have been logged with correct timestamps.
  Pass Criterion: Zero missing entries in the database relative to the total number of expected polling cycles ($\geq 99\%$ data completeness), and all entries retrievable through the dashboard's historical view.
  How this demonstrates the objective: Complete and accurate data persistence over a 72-hour window confirms that the system fulfills the data acquisition and historical logging requirements. The ability to retrieve and display historical data on the dashboard validates the full data pipeline from sensor to screen [5], [6].
- **Test 3.3 – System Uptime (U_{sys}):** Measured over the same 72-hour continuous operation trial to verify sustained availability:

$$U_{sys} = \frac{T_{total} - T_{downtime}}{T_{total}} \times 100\% \tag{16}$$

  where T_{total} is the total trial duration and $T_{downtime}$ is the cumulative time during which sensor polling or the dashboard was unavailable.
  Pass Criterion: $U_{sys} \geq 95\%$. This threshold aligns with IoT industry standards that commonly require at least 95% availability for embedded monitoring systems [18], [26], where continuous sensor coverage is necessary but brief, recoverable interruptions are acknowledged as acceptable. Concretely, 95% uptime over the 72-hour trial permits a maximum cumulative downtime of approximately 3.6 hours—a ceiling beyond which monitoring gaps become operationally significant for farm health management.
  How this demonstrates the objective: A sustained uptime of $\geq 95\%$ confirms that the user interface and data acquisition pipeline are robust enough for continuous farm operation, where downtime directly translates to missed monitoring windows and potential undetected anomalies.
- **Test 3.4 – User Acceptance Testing (UAT) for UI/UX and System Functionality:** The owner or designated caretaker will complete a UAT checklist covering dashboard navigation, interpretation of green/yellow/red status indicators, viewing pen history, editing a feeding schedule, configuring the water refill threshold, acknowledging alerts, checking SMS alert content, and using the Manual Override (RA0) screen to dispense both feed and water on demand. After completing the checklist, the participant will answer a 5-point Likert-scale form.
  Pass Criterion: Mean UAT Likert score ≥ 4.0 out of 5, no critical task failure, and no critical usability item below 3.
  How this demonstrates the objective: UAT verifies that the system is not only functional in engineering terms but also understandable and acceptable to the owner or caretaker who will use it in practice.
- **Test 3.5 – Event Record Completeness for Final Checking:** During final project checking, every scheduled feeding, refill event, threshold exceedance, SMS alert, acknowledgement, manual override, calibration event, and corrective action will be logged with timestamp, module number, measured value, threshold, expected response, actual response, evaluator remarks, and caretaker or owner sign-off.
  Pass Criterion: Event-record completeness $ERC \geq 99\%$, and all required objective-related events are retrievable from the SQLite database or the evaluator checklist.
  How this demonstrates the objective: Complete event records provide traceable evidence that the intended objectives were tested and that the final checking process can verify the system's behavior.

Objective 4: Integrate an SMS module to alert caretakers when the system detects consumption or environmental anomalies.

This objective requires demonstrating that the LoRa-to-SMS notification pipeline reliably detects anomalies and delivers timely SMS alerts to the caretaker's mobile phone.

- **Test 4.1 – Alert Response Accuracy (ARA):** Consumption anomalies will be deliberately induced by withholding or reducing the feed placed in the trough, simulating a sick pig's reduced intake. A total of 20 anomaly events

will be induced across all feeder modules.

$$ARA = \frac{\text{True Positive Alerts}}{\text{Total Induced Events}} \times 100\% \quad (17)$$

Pass Criterion: $ARA \geq 90\%$.

How this demonstrates the objective: A high ARA confirms that the Analytical Engine's rolling-window algorithm reliably detects consumption and environmental anomalies—the prerequisite for any alert to be generated. Without accurate detection, the SMS module cannot fulfill its purpose [13], [14].

- **Test 4.2 – SMS Delivery Rate (SDR):** For each detected anomaly, the system will transmit a LoRa alert to the remote LoRa-GSM gateway, which forwards it as an SMS. Each SMS contains a unique alert identifier for cross-referencing.

$$SDR = \frac{\text{Total SMS Delivered}}{\text{Total Alerts Generated}} \times 100\% \quad (18)$$

Pass Criterion: $SDR \geq 95\%$.

How this demonstrates the objective: An SDR of $\geq 95\%$ confirms that the end-to-end LoRa-to-SMS pipeline reliably delivers alerts to the caretaker's mobile phone. This directly validates the integration of the SMS module as specified in the objective [6], [23].

- **Test 4.3 – End-to-End Alert Latency:** The time elapsed between the Analytical Engine generating an alert and the SMS being received on the caretaker's phone will be measured for each alert event. This encompasses LoRa transmission, gateway processing, and GSM delivery.

  Pass Criterion: Mean latency ≤ 30 seconds per alert.

  How this demonstrates the objective: A delivery latency of ≤ 30 seconds confirms that the SMS alerts are timely enough for the caretaker to take immediate corrective action, validating that the system provides meaningful, real-time remote notification capability [6], [23].

- **Test 4.4 – LoRa Communication Range:** The LoRa link between the main control module and the LoRa-GSM gateway will be tested at increasing distances (e.g., 50 m, 100 m, 200 m, 500 m) within the farm environment. At each distance, 20 alert payloads will be transmitted and the Received Signal Strength Indicator (RSSI) and delivery success rate will be recorded.

  Pass Criterion: $RSSI \geq -120$ dBm and delivery success rate $\geq 95\%$ at the maximum operational distance required by the farm layout.

  How this demonstrates the objective: Confirming adequate LoRa range ensures that the SMS module can function across the actual farm property, validating that the alert system works not just in laboratory conditions but in the intended deployment environment [6], [23].

- **Test 4.5 – Manual Labor Reduction and Control-Group Comparison:** The evaluator will compare the existing manual monitoring process against the system-assisted process. In the control condition, a caretaker records feed level, water availability, air condition, and abnormal observations using the farm's normal manual method. In the system-assisted condition, the caretaker uses the dashboard, historical logs, and SMS alerts. Both conditions will use the same number of pens and checking tasks.

$$MLR = \frac{T_{manual} - T_{system}}{T_{manual}} \times 100\% \quad (19)$$

  Pass Criterion: $MLR \geq 25\%$, equal or fewer missed anomalies in the system-assisted condition, and caretaker Likert score ≥ 4.0 for perceived reduction in routine checking effort.

  How this demonstrates the objective: This test directly measures whether the system reduces manual checking time and improves monitoring support compared with the existing caretaker workflow.

Communication Infrastructure Validation:

- **Test 5.1 – RS-485 Signal Integrity and MODBUS Reliability:** The RS-485 wired bus will undergo signal integrity testing with all feeder nodes active simultaneously. A digital storage oscilloscope will verify that differential voltage levels meet the RS-485 standard minimum of ± 1.5 V. MODBUS RTU packet loss rates will be quantified under simulated electrical noise conditions at the maximum prototype cable length.

  Pass Criterion: Packet loss rate $\leq 1\%$ across all feeder nodes under full bus load.

  How this demonstrates system readiness: While not mapped to a single specific objective, reliable wired communication underpins all four objectives. If the RS-485 bus fails, sensor data cannot reach the central unit, dispensing commands cannot be issued, the dashboard cannot display data, and alerts cannot be generated. This test confirms the foundational infrastructure upon which all objectives depend [5], [6].

M. Deployment Plan

The transition from the laboratory prototype to field operation in a commercial piggery will follow a structured, five-phase deployment strategy to minimize disruption to existing farm operations and to address the need for a local partner for final implementation:

- **Phase 1: Local Partner Identification and Agreement:** The researchers will identify an accessible local piggery owner or farm partner near the proposed deployment area. Before installation, the partner must approve the test schedule, pen access, electrical routing, caretaker participation, UAT forms, and event-recording process. If no partner permits live installation, the project will proceed with partner-observed bench testing and will document the deployment limitation.
- **Phase 2: Site Assessment and Electrical Routing:** An evaluation of the piggery's physical layout will determine the optimal routing for the RS-485 twisted-pair cables and DC power distribution. Cable trajectories will be explicitly planned to avoid electromagnetic interference (EMI) from existing high-voltage farm equipment or large

motors. This phase must produce four deliverables before installation proceeds: a partner confirmation form, a cable routing diagram, a DC power distribution plan, and an EMI risk assessment identifying high-interference zones within the facility.

- **Phase 3: Physical Installation:** The central 0.18 m^3 hopper, auger mechanism, and main control module (Raspberry Pi) will be securely mounted at the centralized feeding station. Individual feeder nodes (ESP32 enclosures, load cells, and level sensors) will be mechanically integrated into the pen structures. The level sensors will be securely mounted directly above the water troughs, and the plumbing will be retrofitted to integrate the automated solenoid valves. Installation is considered complete only after all nodes pass a hardware checklist confirming that all units are powered, all sensors are returning non-zero readings, and all MODBUS device addresses have been successfully assigned and acknowledged by the master.
- **Phase 4: Commissioning, Baseline Initialization, and Threshold Validation:** Once powered, unique MODBUS device addresses will be assigned to each feeder node. The system will initially operate in a "silent monitoring" phase (e.g., 7 to 14 days). During this period, the system will dispense feed and log data to the SQLite database without triggering health alerts, allowing the rolling-window algorithm to establish accurate, pen-specific consumption baselines ($\mu_{t-1,N}$). Anomaly detection will not be activated until a minimum of $N = 14$ previous feeding sessions have been recorded per module. Water-level thresholds will be confirmed with manual depth measurements, while air-quality thresholds will be checked against observed sanitation conditions before and after cleaning or ventilation changes.
- **Phase 5: Caretaker Turnover, UAT, and Final Checking:** Farm operators will undergo practical training on utilizing the Raspberry Pi touchscreen interface. This includes instructing them on how to interpret the time-series charts, modify feeding schedules, address alert logs, and perform routine maintenance such as clearing auger jams or cleaning the optical/gas sensor interfaces. Turnover is considered complete only when each caretaker can independently demonstrate three competencies without assistance: interpreting and responding to a generated alert, modifying a feeding schedule on the dashboard, and performing basic sensor maintenance. The owner or caretaker will then complete the UAT checklist and Likert-scale confidence form, while evaluators record all objective-related events for final project checking.

  This study recognizes that the effectiveness of the system after deployment is influenced not only by its automated feeding, watering, monitoring, and alert functions, but also by the caretaker's ability to operate and maintain the system properly. For this reason, system familiarization and turnover validation will be included to determine whether caretakers can independently interpret alerts, modify feeding schedules, and perform basic maintenance tasks after training.

REFERENCES

[1] OECD and Food and Agriculture Organization of the United Nations (FAO), "Oecd-fao agricultural outlook 2025-2034," tech. rep., OECD Publishing, July 2025.

[2] USDA Foreign Agricultural Service, "Production - pork," 2026.

[3] J. H. B. Dingcong, J. H. Tangoan, W. S. O. Tiu, J. E. L. Undag, R. K. V. Villegas, C. M. I. Gohetia, and J. D. A. Campanera, "Monitoring the feeder and drinker visits of pigs using image processing," in *Department of Computer Engineering*, (Cebu City, Philippines), University of San Carlos, 2017.

[4] "Global, asian, and philippine livestock production systems: A comprehensive sectoral analysis of swine, poultry, and bovine dynamics (2024–2026)," 2026.

[5] M. O. Adeoye, O. T. Arogundade, and T. Olaleye, "Animalia: An iot-based temperature monitoring system for piggery farming," *IRE Journals*, vol. 9, September 2025.

[6] Beyond Technology, "Benefits of smart monitoring in pig farming beyond visual control," June 2025.

[7] J. J. Arguta, J. R. Esteba, J. D. Tuyo, K. J. G. Vizcarra, and P. G. Fernando Jr., "IoP (Internet of Pigs): Design and Implementation of an Automatic Swine Feeder, Waterer and Shower System for a Livestock Multi-Purpose Cooperative," *International Research and Innovation Journal*, vol. 1, no. 1, 2024.

[8] G. K. Dambaulova, V. A. Madin, Z. A. Utebayeva, M. K. Baimyrzaeva, and L. Z. Shora, "Benefits of automated pig feeding system: A simplified cost-benefit analysis in the context of Kazakhstan," *Veterinary World*, vol. 16, no. 11, pp. 2205–2209, 2023.

[9] J. DeRouchey and B. T. Richert, "Feeding systems for swine," Factsheet PIG 09-06-03, Pork Information Gateway, 2010.

[10] F. M. Tangorra and A. Calcante, "Energy consumption and technical-economic analysis of an automatic feeding system for dairy farms: Results from a field test," *Journal of Agricultural Engineering*, vol. 49, no. 3, 2018.

[11] M. Whitney, "Keeping track of water intake to monitor pig performance." University of Minnesota Extension, 2018.

[12] R. Samuel, "Considerations of water for sustainable swine production." SDSU Extension, March 2025.

[13] E. Sadeghi, C. Kappers, A. Chiumento, M. Derks, and P. Havinga, "Improving piglets health and well-being: A review of piglets health indicators and related sensing technologies," *Smart Agricultural Technology*, vol. 5, p. 100246, 2023.

[14] P. Racewicz, A. Ludwiczak, E. Skrzypczak, J. Składanowska-Baryza, H. Biesiada, T. Nowak, S. Nowaczewski, M. Zaborowicz, M. Stanisz, and P. Ślósarz, "Welfare health and productivity in commercial pig herds," *Animals*, vol. 11, April 2021.

[15] A. J. Heber and B. W. Bogan, "Air Quality in Swine Facilities: Managing Ammonia for Health and Compliance," fact sheet, Purdue University Extension, 2019. Recommends maintaining ammonia levels below 25 ppm (OSHA occupational limit) in swine confinement facilities; research indicates concentrations exceeding 20–25 ppm impair respiratory defenses in pigs.

[16] L. D. Jacobson and B. P. Hetchler, "Controlling Ammonia in Swine Facilities," Factsheet PIG 07-05-06, Pork Information Gateway, 2013. Identifies 25 ppm as the occupational threshold limit value (TLV) for ammonia and discusses its use as a practical alarm threshold in swine monitoring systems.

[17] P. Yamsakul, T. Yano, K. Junchum, W. Anukul, and N. Kittiwan, "Machine learning-based detection of pig coughs and their association with respiratory diseases in fattening pigs," *Preprints.org*, June 2025.

[18] E. Carvalho, A. Valente, and E. J. S. Pires, "An IoT-Based Architecture for Precision Livestock Farming: A Systematic Review," *Sensors*, vol. 23, no. 8, p. 3848, 2023. Reviews IoT-based PLF system performance; discusses continuous availability requirements for embedded sensor networks in livestock monitoring, with systems targeting $\geq 95\%$ operational availability as a baseline benchmark for farm deployment.

[19] EveryPig, "Swine management software for data collection," 2026. Accessed: 2026-03-27.

[20] Jyga Technologies, "VISION swine production software," 2026.

[21] R. H. Lo, "Rebooting the philippines swine industry: Challenges, prospects, and policy directions," September 2025.

[22] HealthforAnimals and Food and Agriculture Organization of the United Nations (FAO), "How prevention can reduce the need for antibiotics: Pathways to reduce the need for antimicrobials on farms for sustainable agrifood systems transformation (renofarm)," tech. rep., HealthforAnimals and FAO, Brussels and Rome, 2024.

[23] M. N. Reza, M. R. Ali, M. A. Haque, H. Jin, H. Kyoung, Y. K. Choi, G. Kim, and S.-O. Chung, "A review of sound-based pig monitoring for enhanced precision production," *Journal of Animal Science and Technology*, vol. 67, no. 2, pp. 277–302, 2025.

[24] Y. Zhang, Z. Jiang, Z. Li, H. Qin, J. Wang, and L. Guo, "A Precision Feeding System With Horizontal Screw Feeder Based on ADRC Controller," *International Journal of Swarm Intelligence Research*, vol. 16, no. 1, pp. 1–19, 2025.

[25] P. Chen, T. Huang, B. Wu, H. Qian, F. Xie, B. Liu, D. Liu, and X. Li, "Modeling the Discharge Rate of a Screw Conveyor Considering Hopper–Conveyor Coupling Parameters," *Agriculture*, vol. 14, no. 7, p. 1203, 2024.

[26] J. T. Tibreza, A. Taer, and E. C. Taer, "Enhancing Swine Husbandry: The IoT-Integrated Hog Feeding Assistant," *The Vector: International Journal of Emerging Science Technology and Management*, vol. 32, no. 1, 2023.

[27] X. He, Z. Zeng, Y. Liu, E. Lyu, J. Xia, F. Wang, and Y. Luo, "An Internet of Things-Based Cluster System for Monitoring Lactating Sows' Feed and Water Intake," *Agriculture*, vol. 14, no. 6, p. 848, 2024.

Appendix A
Project Cost Estimate

Proposed Budget for Materials

Item Description	Quantity	Cost/Unit (PHP)	Subtotal (PHP
[1] Raspberry Pi 4/5 (Central Controller)	1	₱4,500.00	₱4,500.00
[2] ESP32 Microcontroller (Edge Feeder Node)	1	₱450.00	₱450.00
[3] 7-inch DSI Touchscreen Display	1	₱2,800.00	₱2,800.00
[4] 0.18 m3 Hopper & Trough Fabrication	1	₱5,000.00	₱5,000.00
[5] Sensors & Actuators (Load Cell, Flow, MQ-135, Solenoids)	1 set	₱2,500.00	₱2,500.00
Total			**₱15,250.00**

Proposed Budget for Lab Equipment/Apparatus Use

Item Description	Quantity	Cost/Unit	Subtotal
[1] Digital Storage Oscilloscope (Phase 2 Testing)	1	₱0.00	₱0.00
[2] Reference Weights & Graduated Cylinders (Phase 1 Testing)	1 set	₱0.00	₱0.00
[3] MQ-135 Calibration Environment	1	₱0.00	₱0.00
Total			**₱0.00**

Room Rental Costs

Room	No. of hours	Cost/hour	Subtotal
[1] University Engineering Lab	—	₱0.00	₱0.00
[2] Piggery Field Site (Site Assessment Phase)	—	₱0.00	₱0.00
Total			**₱0.00**

Miscellaneous Costs

Manpower/Overtime Costs

Name of Person	Position	Rate/Fee	No. of Hours
[1] Manpower (Overtime Installation)	Lead/Helper	₱0.00	—
Total			

External Analysis

Description	Samples	Analysis Fee	Subtotal
[1] Ammonia Threshold Calibration (Known Concentrations)	1 Session	₱1,500.00	₱1,500.00
Total			

Documentation Costs

Description	Quantity	Cost/Unit	Subtotal
Thesis Manuscript Printing and Binding	5	₱300.00	₱1,500.00
Total			**₱1,500.00**

Transportation and Accommodation Costs

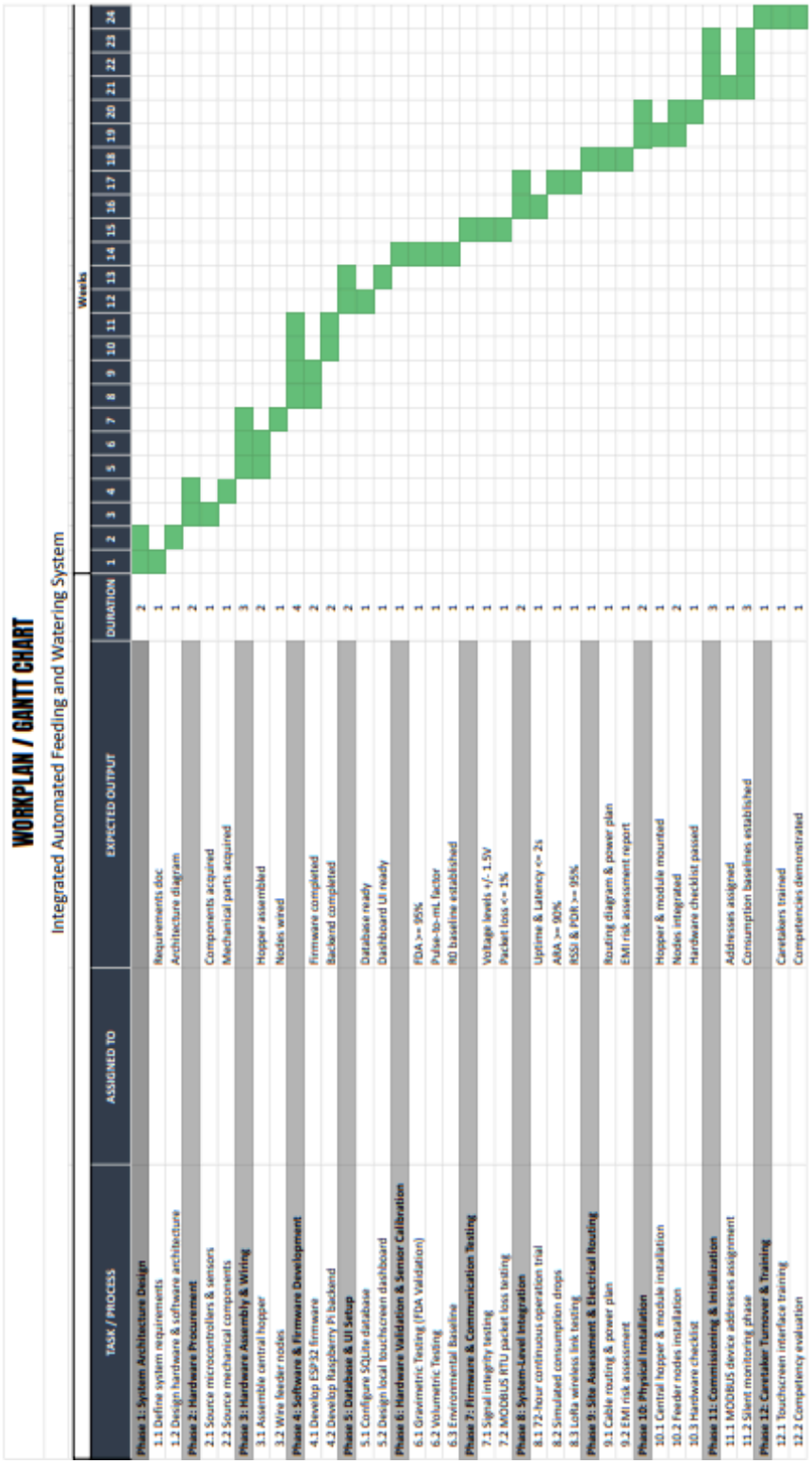
Description	No. of persons	Rate/person	Subtotal
Field Site Visit (Cagayan, Round Trip)	5	₱5,060.00	₱25,300.00
Total			**₱25,300.00**

Incidental Costs

Description	Quantity	Cost/Unit	Subtotal
RS-485 Cables & DC Power Distribution	1 lot	₱1,000.00	₱1,000.00
Total			**₱1,000.00**

GRAND TOTAL	**₱42,150.00**

Appendix B
Workplan and Expected Outputs



Appendix C
Notice of Adviser Acceptance

UNIVERSITY of SAN CARLOS
SCIENTIA • VIRTUS • DEVOTIO

NOTICE OF ACCEPTANCE

We, the undersigned, hereby agree to collaborate, as part of the requirements of the Department of Computer Engineering's **BS CPE thesis courses**, in a project/research study entitled, **Integrated Automated Feeding and Watering System with Centralized Data Monitoring for Commercial Piggeries**. This Notice of Acceptance is effective beginning the **1st Semester of AY 2024-2025**.

The **students** will prepare a project/research proposal, including an introductory chapter, a review of related literature, methodology, and a proposed timetable and budget. Upon approval, the students will conduct the study as outlined in the approved methodology, document the results, discussion, conclusions, and recommendations, and defend the study before an appointed thesis evaluation committee.

The **adviser**, whose own research project(s) may provide the foundation for the students' study, is responsible for guiding the students through the project/research study, holding regular consultation sessions at a mutually agreed frequency (e.g., weekly, bimonthly), and reviewing and critiquing the thesis manuscript before it is submitted to the thesis evaluation committee for the proposal hearing and oral examination.

The **assistant adviser** will also support the students within his/her technical capacity and expertise. In the event of the adviser's absence or any circumstance that precludes the adviser from fulfilling their duties, the assistant adviser shall assume the responsibilities of thesis advising.

To formalize this agreement, we hereby affix our signatures below.

PACE, VAUN MICHAEL C.

PARRAS, PAUL ANDREW F.

SORIANO, LORENZ ANTHONY L.

ENGR. ALVIN JOSEPH S. MACAPAGAL
Thesis Adviser

ENGR. JAN DAVE Q. CAMPAÑERA
Thesis Assistant Adviser

Witnesses:

DR. GODWIN S. MONSERATE
CPE 3107 Coordinator

DR. ANTONIETTE M. CAÑETE
Chair, Department of Computer Engineering

USC - Talamban Campus
Gov. M. Cuenco Ave., Talamban, Cebu City, Philippines 6000
Email: comedept@usc.edu.ph
www.usc.edu.ph

Appendix D
Student-Adviser Thesis Counseling Logbook

UNIVERSITY of SAN CARLOS

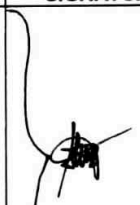
Department of Computer Engineering
School of Engineering

STUDENT-ADVISER THESIS COUNSELING LOGBOOK

✓Thesis Proposal (CPE 3207L) Thesis Implementation (CPE 3301L | CPE 4101L)

1st Semester AY _____ ✓2nd Semester AY 2025-2026 Summer _____

GROUP LETTER:	**V**
PROJECT TITLE:	**Integrated Automated Feeding and Watering System with Centralized Data Monitoring for Commercial Piggeries**
NAME OF ADVISER:	**Engr. Alvin Joseph Macapagal**
NAME OF ASST. ADVISER:	**Engr. Jan Dave Q. Campañera**
NAME OF STUDENTS:	**Vaun Michael C. Pace**
	Paul Andrew F. Parras
	Lorenz Anthony L. Soriano

DATE	VENUE / MEDIUM	TIME	TOPICS DISCUSSED	ADVISER'S / ASST. ADVISER'S SIGNATURE
12/04/2025	DML	10:15 AM - 10:30 AM	-Presented a thesis proposal, and the adviser recommended securing a validating client and identifying a gas sensor suitable for detecting swine emissions.	
12/17/2025	DML	2:00 PM - 3:00 PM	-Discussed with the adviser about conducting a feasibility study for the clients.	

USC - Talamban Campus
Nasipit, Talamban, Cebu City, Philippines 6000
Email: comedept@usc.edu.ph
www.usc.edu.ph

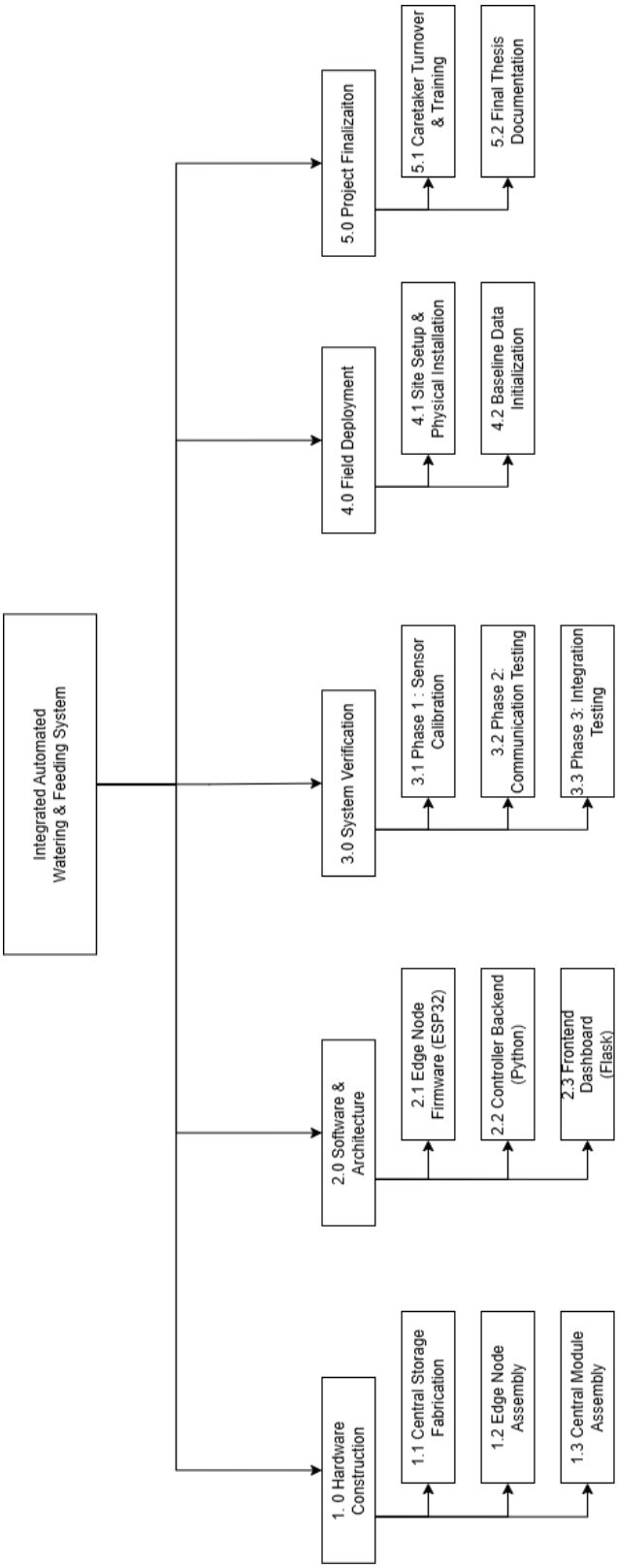
UNIVERSITY of SAN CARLOS

SCIENTIA • VIRTUS • DEVOTIO

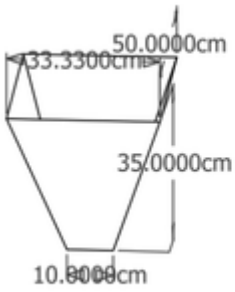
03/18/2025	DML	1:30 PM - 2:30 PM	-Begin drafting, obtain a thesis copy from Engr. Campañera. -Confirm details with the client -Review related papers to identify common elements.	
03/25/2025	DML	1:30 PM - 2:10 PM	-Clearly qualify the problems, formulate the objectives, structure the manuscript based on the required format. -Focus on completing the Introduction, Significance of the Study, and Goals and Objectives sections.	
04/08/2025	DML	1:30 PM - 2:00 PM	- Develop the methodology -Restructure the goals and objectives -Decide on testing conditions, preferably validating the system with pigs.	
04/15/2025	DML	2:00 PM - 2:15 PM	-changes to the methodology based on the adviser's recommendations	
04/27/2025	DML	1:00 PM - 1:20 PM	-Finalizing the manuscript	

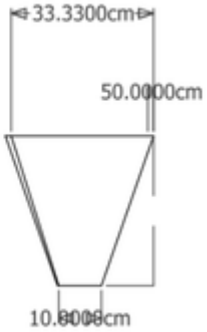
USC - Talamban Campus
Nasipit, Talamban, Cebu City, Philippines 6000
Email: comedept@usc.edu.ph
www.usc.edu.ph

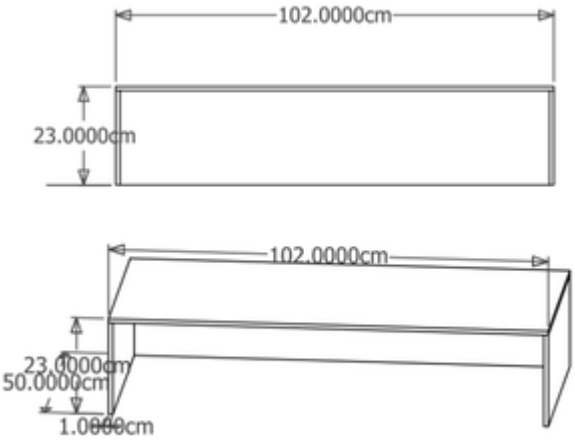
Appendix E
Work Breakdown Structure

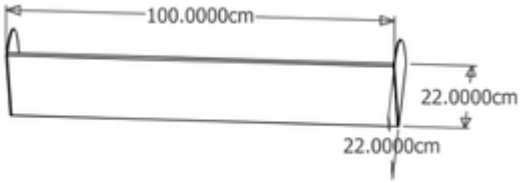


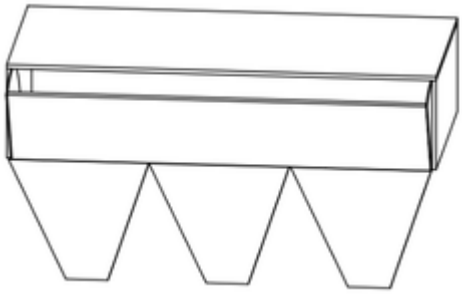
Appendix F
Prototype Design

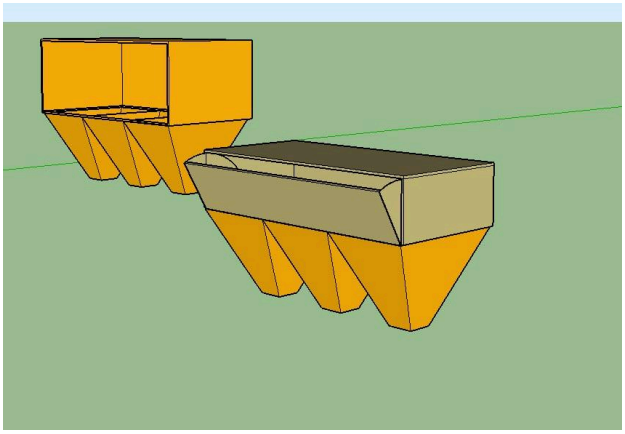


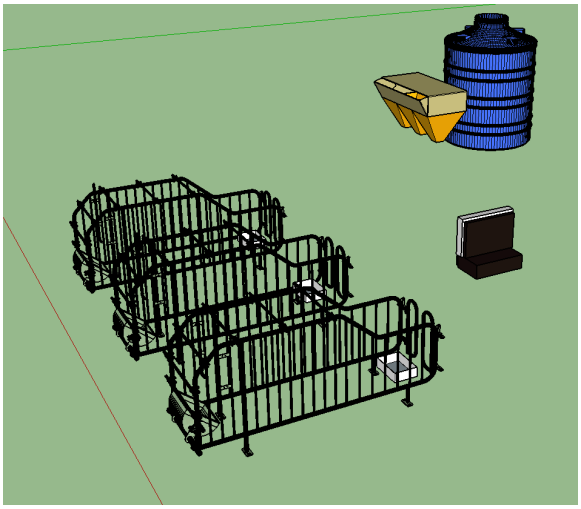




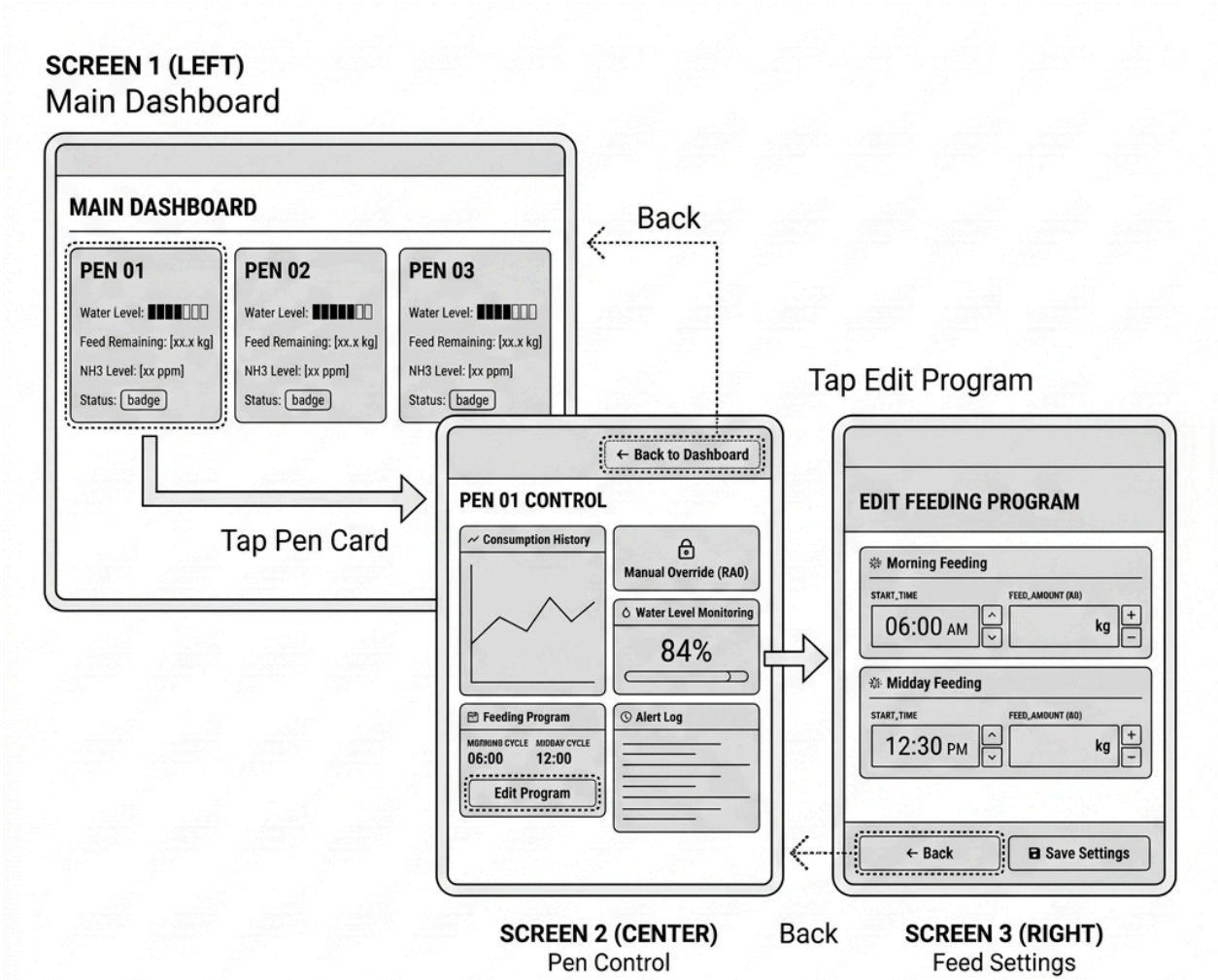








Appendix G
UI Wireframe



Appendix H
Project Deliverables

#	Project Deliverables	Status
A. Hardware Deliverables		
1	Central Control Module (Raspberry Pi, Touchscreen, RS-485, LoRa, UPS)	Ongoing
2	Central Feed Hopper Assembly (0.18 m^3 Frustum Hopper with Guillotine Valve)	Ongoing
3	Auger Conveyor Mechanism (NEMA 17 Stepper Motor-Driven Feed Dispenser)	Ongoing
4	Feeder Module Assemblies ×3 (ESP32, HX711, Load Cell, MQ-135, JSN-SR04T)	Ongoing
5	Feed Trough Units ×3 (with Load Cell Mounts)	Ongoing
6	Water Trough & Solenoid Valve Assembly ×3 (PVC-Piped Supply Tank)	Ongoing
7	RS-485 Wired Bus Network (Twisted-Pair, MAX485 Transceivers)	Ongoing
8	LoRa-GSM Remote Alert Gateway	Ongoing
9	UPS Power Backup Module (18650 Lithium Cell Circuit)	Ongoing
10	Manual Override Push Button (Active-High, Pin RA0)	Ongoing
11	Standard Reference Weights (for Load Cell Calibration)	Ongoing
B. Software Deliverables		
12	ESP32 Edge Node Firmware (FreeRTOS Dual-Core, MODBUS RTU Slave)	Ongoing
13	Raspberry Pi Python Backend (PyModbus Poller, Command & Alert Services)	Ongoing
14	Anomaly Detection / Analytical Engine (Rolling-Window, N=14 Sessions)	Ongoing
15	SQLite Local Database (Timestamped Telemetry, Alerts, Schedules)	Ongoing

#	Project Deliverables	Status
16	Flask Web Application Backend (RESTful API, AJAX Polling)	Ongoing
17	Touchscreen Dashboard Frontend (Main Dashboard, Pen Control, Feed Settings)	Ongoing
18	LoRa-SMS Alert Pipeline (Message Queuing, LoRa Payload Encoding)	Ongoing
19	Python Watchdog Service (Heartbeat Logging, Uptime Computation)	Ongoing
C. Testing and Evaluation Deliverables		
20	Feed Dispensing Accuracy Report – T6.1 (FDA ≥ 95%, 30 Trials/Module)	Ongoing
21	Water Level Sensor Validation Report – T6.2 (±1 cm, 10 Trials)	Ongoing
22	MQ-135 Air Quality Calibration Report – T6.3 (25 ppm NH_3 Threshold)	Ongoing
23	RS-485 Signal Integrity Report – T7.1 (Differential Voltage ≥ ±1.5 V)	Ongoing
24	MODBUS RTU Packet Loss Report – T7.2 (Packet Loss ≤ 1%)	Ongoing
25	System Uptime & Data Latency Report – T8.1 (Usys ≥ 95%, Latency ≤ 2 s)	Ongoing
26	Alert Response & SMS Delivery Report – T8.2 (ARA ≥ 90%, SDR ≥ 95%)	Ongoing
27	LoRa Communication Range Report – T8.3 (RSSI ≥ −120 dBm at Max Distance)	Ongoing
28	Load Cell Calibration & Accuracy Report - T2.1 (R^2 ≥ 0.99, ±2% accuracy)	Ongoing
29	End-to-End Alert Latency Report - T4.3 (Mean latency ≤ 30 seconds per alert)	Ongoing
D. Deployment Deliverables		
30	Site Assessment & Electrical Routing (Cable Routing Diagram, EMI Report)	Ongoing
31	Physical Installation (Hopper, Feeder Nodes, Solenoid Plumbing)	Ongoing